

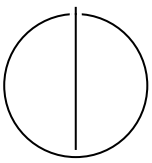
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

File-Layout-Aware Caching for Cloud-Native Analytical Query Engines

Samuel von Khuon-Wildegg





SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

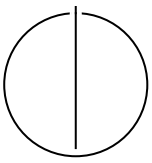
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

File-Layout-Aware Caching for Cloud-Native Analytical Query Engines

Dateilayout-basiertes Caching für cloudnative analytische Query-Engines

Author:	Samuel von Khuon-Wildegg
Examiner:	Prof. Dr. Pramod Bhatotia
Supervisor:	M.Sc. Ilya Meignan–Masson
Submission Date:	21.09.2026



Acknowledgments

I would like to thank Prof. Dr. Pramod Bhatothia for giving me the opportunity to write this thesis at his chair. I am grateful for the opportunity to work on this topic and for the support and environment provided throughout the thesis.

I would especially like to thank my advisor, M.Sc. Ilya Meignan–Masson, for the guidance and support throughout this work. I am particularly grateful for our weekly meetings, which provided valuable feedback, helped me overcome challenges, and kept the work on track. These meetings were a great help throughout the entire thesis.

Finally, I would like to thank my family for their support throughout my entire time as a student. Their encouragement and support have accompanied me throughout my studies and, in particular, during the writing of this thesis. I am very grateful for everything they have done for me during this time.

Abstract

Modern cloud-native database management systems increasingly separate compute and storage, allowing both resources to be scaled independently. However, this architecture requires data to be retrieved from remote persistent storage, such as cloud object stores, making storage accesses more costly in terms of latency and data transfer. Efficient caching is therefore essential to reduce the number of accesses to remote storage and improve query performance.

This thesis investigates file-layout-aware caching techniques for cloud-native analytical query engines. Instead of using conventional fixed-size cache blocks, Parquet files are cached based on their column chunks, allowing the cache to store data at a granularity that corresponds more closely to the data actually required by analytical queries. Since column chunks have different sizes, this approach introduces the need for cache replacement policies that account for the size of individual cache blocks. Several size-aware variants of established replacement policies are therefore implemented and evaluated alongside conventional replacement policies.

The evaluation using TPC-H workloads shows that, under the evaluated configurations, column chunk-based caching can improve effective cache utilization and reduce query execution time. The results further show that incorporating block size into cache replacement decisions can improve cache performance for workloads with variable-sized cache blocks. Combining column chunk-based caching with size-aware S3-FIFO replacement reduces total execution time by approximately 46% across the 20 evaluated five-query sequences compared to a conventional fixed-size cache using Least Recently Used (LRU) replacement, with individual workload speedups of up to 4.87×.

Contents

Acknowledgments iii

Abstract iv

1 Introduction 1

2 Background 3
- 2.1 Analytical databases . . . 3
- 2.2 Parquet Files . . . 3
- 2.3 Caching . . . 4
  - 2.3.1 Block Size . . . 5
  - 2.3.2 Replacement Policy . . . 5
- 2.4 Analytical Query Engines in the Cloud . . . 6
- 2.5 New Challenges . . . 7

3 Overview 9
- 3.1 Dynamic Cache Blocks . . . 9
- 3.2 Size-Aware Policies . . . 9

4 Design 11
- 4.1 Column Chunk-Based Caching . . . 11
- 4.2 Replacement Policies . . . 12
  - 4.2.1 Interchangeable Replacement Policies . . . 12
  - 4.2.2 Size-Aware Replacement Policies . . . 13

5 Implementation 14
- 5.1 DuckDB . . . 14
- 5.2 Dynamic blocks . . . 15
- 5.3 Replacement policies . . . 17
  - 5.3.1 General Replacement Policy Interface . . . 17
  - 5.3.2 LRU Policy . . . 19
  - 5.3.3 Size-Aware LRU Policy . . . 20
  - 5.3.4 S3-FIFO Replacement Policy . . . 21

5.3.5 Size-Aware S3-FIFO Replacement Policies 24

6 Evaluation **28**
6.1 Evaluation Setup 28
6.2 Simulation of Remote Storage Latency 28
6.3 Benchmark Parameters 29
6.4 Dynamic block size 30
6.4.1 Dynamic Block Sizes on Initial Load 30
6.4.2 Improved Cache Memory Utilization 37
6.4.3 Evaluation on Query Sequences 39
6.5 Size-aware Replacement Policies 40
6.5.1 LRU vs. Size-Aware LRU 40
6.5.2 LRU vs. S3-FIFO 43
6.5.3 S3-FIFO vs. Size-Aware S3-FIFO 43
6.6 Overall Comparison 46

7 Related work **53**
7.1 Caching with Variable-Sized Blocks 53
7.2 Replacement Policies Beyond LRU 53

8 Summary **55**

9 Future work **57**

Abbreviations **59**

List of Figures **60**

List of Tables **63**

Bibliography **64**

1 Introduction

Cloud-native database management systems increasingly separate compute and storage. Instead of storing analytical data directly on the servers responsible for query processing, data can be kept in remote persistent storage systems such as cloud object stores. This separation allows compute and storage resources to be scaled independently and makes it possible to allocate compute resources according to the current workload. However, it also means that query execution will require data to be retrieved from remote storage. Compared to accessing data from local memory or storage, these accesses can introduce significant latency and data transfer costs.

Caching is therefore an important component of cloud-native analytical database systems. Frequently accessed data can be kept closer to the compute nodes so that subsequent accesses can be served without contacting the remote storage system. The effectiveness of such a cache depends not only on its size but also on how data is divided into cache entries and how entries are selected for eviction when the cache becomes full. Traditional caching approaches commonly divide data into fixed-size blocks. This provides a simple and predictable cache organization, but the fixed block boundaries do not necessarily correspond to the structure of the data being accessed by an analytical query.

Columnar file formats such as Parquet [Apa26] provide a more suitable structure for analytical workloads. A Parquet file is organized into row groups and column chunks, allowing individual columns to be stored and accessed independently. Analytical queries frequently access only a subset of the columns contained in a file. Caching according to the layout of the underlying Parquet file can therefore allow the cache to store the data at a granularity that more closely matches the data requested by the query. Instead of loading arbitrary fixed-size regions, the cache can store individual column chunks that correspond to meaningful units within the file.

However, using the file layout as the basis for cache entries introduces a new challenge. Unlike fixed-size blocks, Parquet column chunks can have substantially different sizes. Consequently, treating every cache entry equally can result in inefficient use of the available cache capacity. For example, evicting a large column chunk and a small column chunk after the same number of accesses does not account for the different amount of cache space occupied by the two entries. Replacement policies therefore need to consider the size of individual cache entries when operating on variable-sized

blocks.

This thesis investigates a file-layout-aware caching approach for cloud-native analytical query engines. The approach uses Parquet column chunks as cache entries instead of relying exclusively on fixed-size blocks. To address the resulting variable block sizes, several size-aware variants of cache replacement policies are implemented. These policies modify the replacement behavior based on the size of individual cache entries while retaining the general principles of their underlying policies. In addition, different replacement policies can be used interchangeably, allowing their behavior to be evaluated under different analytical workloads and cache configurations.

The proposed approach is implemented in DuckDB [RM19] and evaluated using TPC-H [Tra26] workloads. The evaluation considers both the effect of using Parquet column chunks instead of fixed-size blocks and the effect of incorporating block size into cache replacement decisions. The results show that file-layout-aware caching can reduce the amount of data that needs to be transferred from persistent storage and can improve cache utilization. They further demonstrate that size-aware replacement policies can provide additional benefits when the cache contains variable-sized entries. Combining both techniques can substantially reduce query execution time compared to a conventional fixed-size cache using LRU replacement.

The remainder of this thesis is structured as follows. Chapter 2 introduces the background relevant to cloud-native analytical database systems, caching, Parquet, and cache replacement policies. Chapter 3 provides an overview of the proposed approach. Chapter 4 describes the design of the proposed caching approach and the evaluated replacement policies. Chapter 5 presents the implementation of the proposed approach. Chapter 6 presents the experimental setup and evaluates the different caching and replacement strategies. Chapter 7 discusses related approaches to variable-sized caching and cache replacement. Chapter 8 summarizes the results of the thesis. Finally, Chapter 9 discusses possible directions for future work.

2 Background

2.1 Analytical databases

The increasing volume, variety, and velocity of data generated by modern applications have made efficient data analysis a central challenge for organizations. Analytical databases play an important role in addressing this challenge because they are specifically designed to store and process large datasets and to execute complex, read-intensive queries efficiently. Unlike transactional databases, which primarily support frequent updates and short operations, analytical databases are optimized for aggregations and large-scale data scans, often using techniques such as columnar storage, compression, and parallel query execution. This enables organizations to transform large amounts of raw data into actionable insights and supports applications such as business intelligence, forecasting, and data-driven decision making. As datasets continue to grow, the ability of analytical databases to efficiently scan, aggregate, and analyze large amounts of data becomes increasingly important.

2.2 Parquet Files

The data processed by analytical queries has to be persistently stored in a suitable format. The choice of storage format influences how data can be organized, accessed, and processed by an analytical database system. Parquet [Apa26] has become a cornerstone storage format for large-scale analytical systems [Liu+25] and is therefore the format considered in this thesis.

Parquet is a column-oriented file format designed for the efficient storage and processing of large analytical datasets. Unlike row-oriented formats, Parquet stores the values of each column together, allowing analytical query engines to read only the columns required by a query. A Parquet file is organized into row groups, each containing a subset of the rows, and each row group is further divided into column chunks, with one column chunk for each column, as illustrated in Figure 2.1. Column chunks are further divided into smaller pages, which contain the encoded and optionally compressed data.

In addition to the actual data, Parquet files contain metadata such as the number

of rows, data locations, encodings, and column statistics. These statistics can be used by query engines to skip row groups that cannot contain relevant data, reducing the amount of data that needs to be read. The combination of columnar storage, compression, and metadata makes Parquet well suited for analytical workloads and large-scale data processing systems.

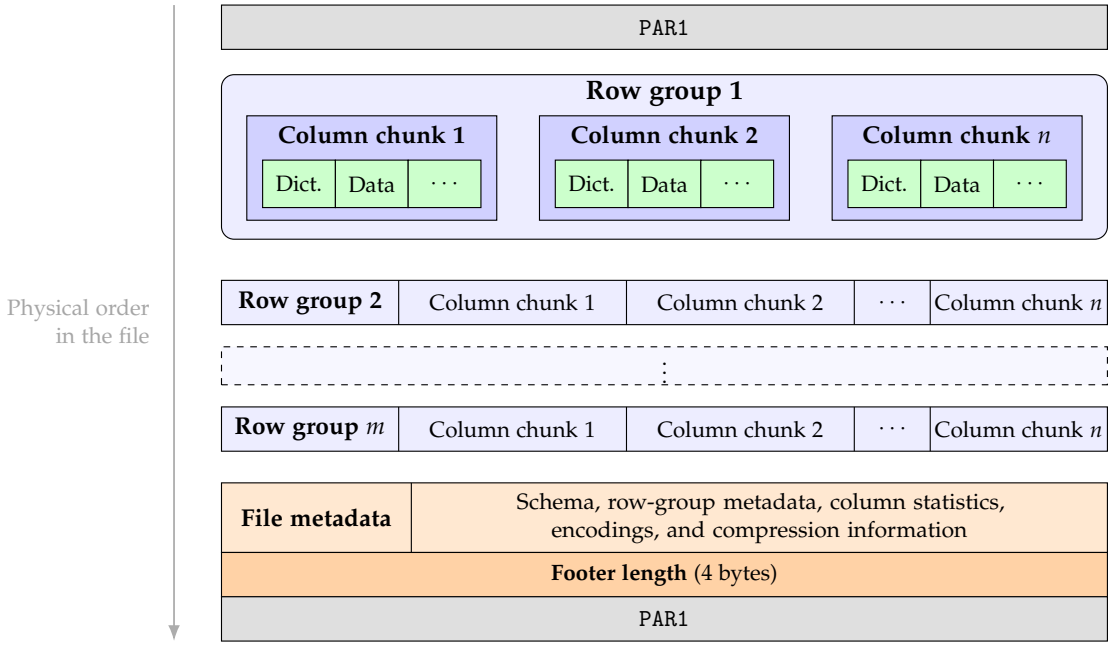


Figure 2.1: Overview of the physical structure of an Apache Parquet file. The file consists of row groups containing one column chunk per column. Each column chunk is divided into encoded and compressed pages. The file metadata is stored in the footer.

2.3 Caching

The large datasets processed by analytical queries are typically stored on persistent storage rather than in main memory. Persistent storage provides substantially larger capacity, but its access latency and bandwidth are orders of magnitude worse than those of main memory. Consequently, repeatedly accessing data directly from persistent storage can significantly limit query performance. To reduce the cost of these accesses, it is common to introduce a cache layer between the query engine and persistent storage. The cache stores recently or frequently accessed data in faster storage so that

subsequent accesses can be served without accessing the persistent storage again.

When a query accesses data that is already present in the cache, the access results in a cache hit and the data can be served directly from the cache. If the requested data is not present, a cache miss occurs and the data has to be retrieved from persistent storage before it can be used. The effectiveness of such a cache therefore depends on both the amount of data that can be stored and how the cached data is organized and managed.

Two important aspects of cache management are the size of the individual cache blocks and the policy used to determine which blocks are removed when space is required. The following subsections discuss these two aspects and describe how they are commonly handled in existing database systems.

2.3.1 Block Size

Most cache architectures are agnostic to the format of the data they cache. This is a natural design choice because database systems may support multiple persistent storage formats, and making the caching layer aware of each format would require format-specific implementations. Instead, caches commonly divide the accessed data into fixed-size blocks that can be managed independently of the underlying data format.

Using fixed-size blocks provides a simple and general interface between the storage layer and the cache. The cache does not need to understand how the data is organized within a file and can apply the same mechanisms to different storage formats. However, this also means that the cache boundaries do not necessarily correspond to meaningful data boundaries defined by the underlying storage format. As a result, a cache block may contain data that is not required by a query, while data that naturally belongs to a single logical unit in the storage format may span multiple cache blocks.

2.3.2 Replacement Policy

If the cache is large enough to hold the entire dataset being accessed, every data block only needs to be fetched from persistent storage the first time it is accessed. Subsequent accesses to the same data can then be served directly from the cache, resulting in cache hits. However, this situation is uncommon in practice, as analytical datasets will most likely be significantly larger than the available cache capacity.

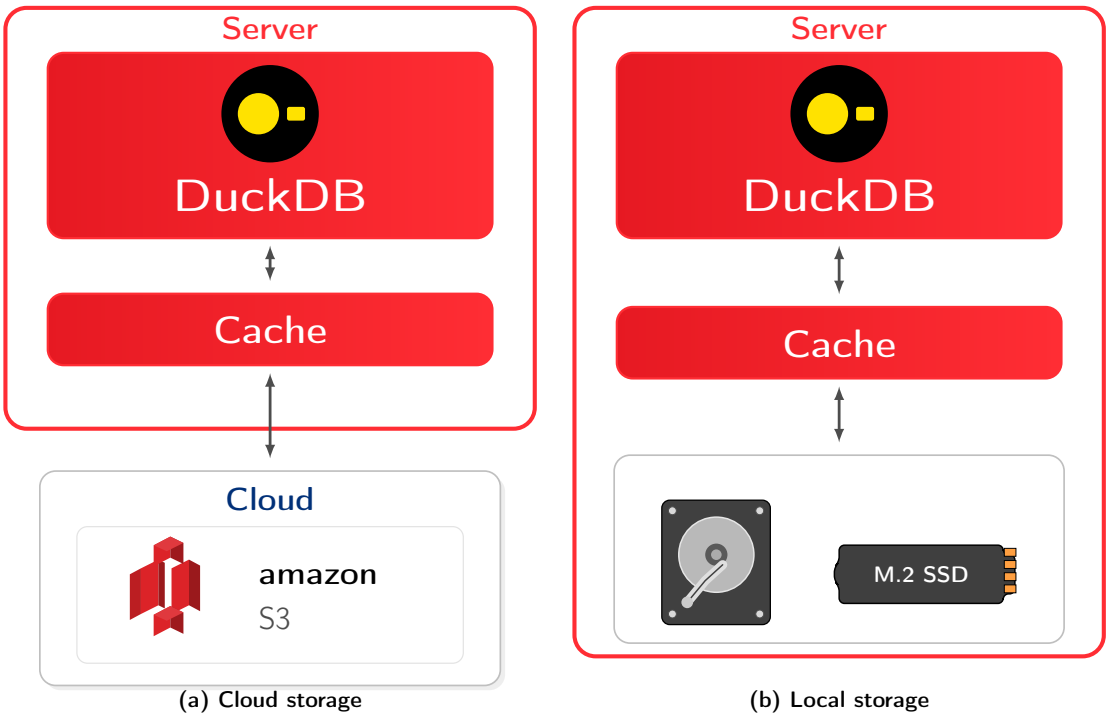
Once the cache is full and a previously uncached block needs to be loaded, space has to be made available by evicting a block that is already stored in the cache. This is where the cache replacement policy becomes important. An effective replacement policy should select a block that is unlikely to be accessed again in the near future. By retaining blocks that are more likely to be reused, the policy can increase the number of future cache hits and reduce the number of expensive accesses to persistent storage.

Many database systems use relatively simple replacement policies, such as LRU, or mechanisms that approximate its behavior [Sta26]. One reason for this is that such policies are comparatively simple to implement while providing reasonable performance across a wide range of general-purpose workloads. However, no replacement policy is optimal for every workload. If the characteristics of a workload are known in advance, using a replacement policy specifically suited to that workload can potentially improve cache performance further.

2.4 Analytical Query Engines in the Cloud

Traditional Database Management System (DBMS) architectures typically combined data storage and query processing on the same server. As cloud-native analytical systems have become increasingly prevalent, this architecture has shifted towards a separation of compute and storage [Yu25]. In this model, compute nodes are responsible for processing queries, while the data is stored separately in remote storage systems such as Amazon S3 [Ama26]. Figure 2.2 illustrates the difference between the traditional architecture and the cloud-native approach.

The separation of compute and storage allows both resources to be scaled independently. This can improve resource utilization because compute resources do not need to remain tied to a particular dataset. For example, if a compute node is idle, it can be used to process queries for a different dataset without requiring the corresponding data to be stored locally. This flexibility allows resources to be allocated according to the current workload and avoids the need to provision storage and compute capacity together.



(a) Cloud storage (b) Local storage

Figure 2.2: DuckDB caching architectures using cloud and local storage.

2.5 New Challenges

While the decoupling of compute and storage offers a new level of flexibility, it also comes with a significant drawback. The tight coupling of the traditional architecture meant that storage and compute were close to each other from a systems perspective. A comparatively inexpensive read from a local disk can become an expensive network transaction when the data is stored remotely (see Figure 2.3).

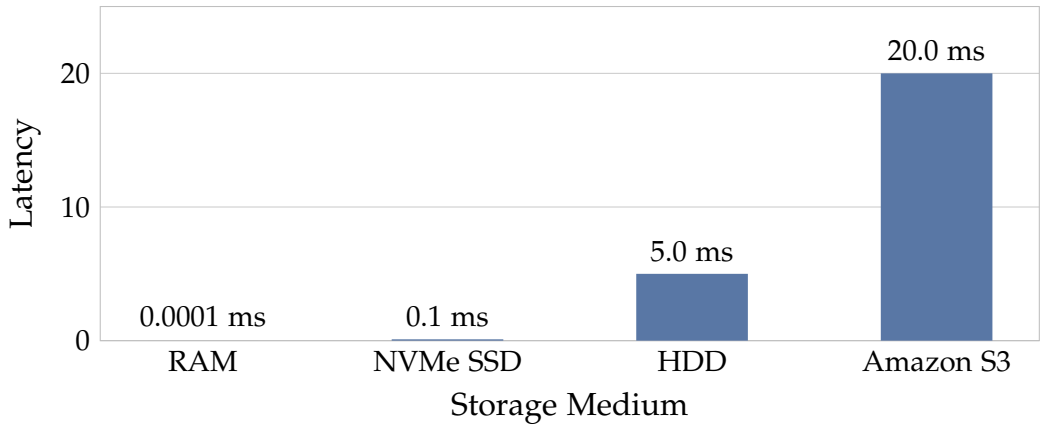


Figure 2.3: Approximate access latency for different storage media.

Even in traditional architectures, data read from persistent storage is commonly cached in Random-Access Memory (RAM) so that subsequent accesses can be served without additional reads from the storage device. With the disaggregation of storage and compute, however, the cost of accessing persistent storage can increase by orders of magnitude. Avoiding unnecessary reads from persistent storage is therefore more important than ever.

To address this increased cost, this thesis revisits two aspects of conventional caching mechanisms with the goal of improving cache efficiency. In particular, we investigate whether the granularity of caching can be aligned with the structure of the underlying data and how cache replacement can be adapted to the resulting variable-sized cache blocks. These considerations are captured by the following two research questions:

1. Can cache efficiency be improved by using the data layout of Parquet files as the granularity for caching?

2. How can established cache replacement policies be adapted to account for the varying sizes of cache blocks resulting from this approach?

3 Overview

The proposed approach addresses the two challenges identified in the previous section: the granularity at which data is cached and the management of cache entries with different sizes. The first part of the approach replaces the conventional fixed-size cache blocks with blocks derived from the layout of the underlying Parquet files. The second part provides an interchangeable replacement-policy interface that allows different policies to be evaluated and adapted to the resulting variable-sized cache blocks.

3.1 Dynamic Cache Blocks

Conventional caches divide data into fixed-size blocks independently of the structure of the underlying file. In contrast, the proposed approach allows cache blocks to be derived from the layout of a Parquet file. In particular, Parquet column chunks are used as the cache granularity. A column chunk represents the data of one column within a row group and therefore provides a naturally defined unit of data access for analytical queries.

Using column chunks as cache blocks results in variable-sized cache entries. The size of a column chunk depends on factors such as the column data and its encoding and compression, and can therefore differ substantially between chunks. This allows the cache to store data in units that more closely correspond to the data accessed by queries, but also requires the cache to handle blocks whose sizes are no longer uniform.

The approach retains support for conventional fixed-size blocks. This makes it possible to use the existing fixed-size caching mechanism when the layout of a file is not available or when data cannot be mapped to a column chunk. Consequently, the use of dynamically sized blocks does not require all cached data to follow a single format-specific mechanism.

3.2 Size-Aware Policies

The introduction of variable-sized cache blocks also affects cache replacement. Conventional replacement policies generally operate on cache entries without considering

differences in their sizes. To investigate how replacement can be adapted to variable-sized blocks, the cache is designed around an interchangeable replacement-policy interface. The cache management itself is therefore separated from the logic responsible for determining which entry should be removed.

The interface allows different replacement policies to be implemented and evaluated without changing the underlying cache mechanism. Established policies such as LRU and S3-FIFO [Yan+23] were implemented using this interface as reference policies. In addition, size-aware variants of these policies were developed to investigate how the size of a cache entry can be incorporated into replacement decisions.

The size-aware policies modify the replacement behavior to account for the different sizes of dynamically sized cache blocks. This enables the cache to consider not only the access history of an entry but also the amount of cache capacity occupied by that entry. The interchangeable policy interface makes it possible to compare these size-aware variants with their conventional counterparts within the same caching architecture.

4 Design

4.1 Column Chunk-Based Caching

The first design change replaces the exclusive use of fixed-size cache blocks with the ability to cache Parquet column chunks as individual cache entries. Traditional block-based caching divides the input data into equally sized blocks and caches the blocks that contain the requested data. While this approach is independent of the underlying file format, it does not take advantage of the structure of Parquet files. Since analytical query engines frequently access individual columns, caching data according to the column chunk structure can provide a cache granularity that is better aligned with the data accessed by the DBMS.

In the proposed design, tables that should use column chunk-based caching can be declared before query processing begins. For these tables, the cache obtains the relevant information about the column chunks contained in their Parquet files. When the DBMS subsequently issues a read request, the cache maps the requested byte range to the corresponding column chunk. Instead of caching only the fixed-size block containing the requested bytes, the cache loads and stores the complete column chunk as a single cache entry.

Caching complete column chunks ensures that cache entries correspond directly to the physical structure of the Parquet file. Once a column chunk has been cached, subsequent reads that access data contained within the same chunk can be served from the cached entry. The size of cache entries is therefore no longer fixed but depends on the size of the corresponding column chunk.

The column chunk-based caching mechanism is designed to coexist with the existing fixed-size caching mechanism. Not every read request can necessarily be mapped to a known column chunk. For example, the accessed file may not belong to a table declared for column chunk-based caching, or the requested byte range may not correspond to a column chunk known to the cache. In these cases, the request is handled by the existing mechanism and the requested data is cached using fixed-size blocks.

This fallback behavior preserves the general applicability of the cache while allowing selected tables to benefit from a cache granularity based on the Parquet file structure. As a result, the design supports both cache organizations within the same system: tables that are explicitly configured for column chunk-based caching use variable-sized cache

entries corresponding to Parquet column chunks, while all other requests continue to use the existing fixed-size block caching mechanism.

4.2 Replacement Policies

4.2.1 Interchangeable Replacement Policies

The effectiveness of a cache replacement policy strongly depends on both the workload characteristics and the available cache capacity. Consequently, no single replacement policy performs optimally for all combinations of workloads and cache sizes. Making the replacement policy interchangeable therefore allows the cache to be adapted to the workload instead of assuming that one policy is universally suitable.

For example, LRU is generally effective for workloads with temporal locality, where data accessed recently is likely to be accessed again in the near future. Such access patterns occur when queries repeatedly access similar data or when multiple queries share parts of their working sets. In these cases, retaining recently accessed cache entries can significantly increase the cache hit rate and reduce expensive accesses to remote storage.

However, other workloads exhibit substantially different behavior. A scan-heavy workload may sequentially access a large amount of data with little or no reuse. If the cache is smaller than the workload's working set, a policy such as LRU can perform poorly because entries inserted during the scan may evict previously cached data before it can be reused. As a result, the cache experiences many misses while continuously replacing entries that provide little benefit. In such scenarios, another replacement policy may retain useful data more effectively and achieve better performance.

The most effective policy can therefore change not only between different workloads but also as the cache size changes. A policy that performs well when the cache can hold a large fraction of the working set may perform considerably worse when the cache capacity is reduced. Similarly, increasing the cache size can change the amount of temporal locality that a policy is able to exploit and consequently alter the relative performance of different policies.

For this reason, the cache design separates cache management from the replacement policy itself and provides a mechanism for selecting or exchanging policies. This enables different policies to be evaluated under identical conditions and allows the cache configuration to be adapted to the expected workload and available memory. Such modularity is particularly important for analytical query engines where workloads can range from repeated selective queries with strong temporal locality to large sequential scans with limited reuse.

4.2.2 Size-Aware Replacement Policies

While making cache replacement policies interchangeable allows the cache to adapt to different workload characteristics, the use of Parquet column chunks as cache entries introduces an additional challenge. Unlike a traditional block cache, where all cache blocks typically have the same fixed size, Parquet column chunks can vary considerably in size. Consequently, eviction decisions should not only consider which entries are accessed frequently or recently, but also how much cache capacity each entry occupies.

For example, evicting a single large column chunk may free enough space to admit several smaller entries, while evicting multiple small entries may provide less benefit despite removing data that could have been reused. A replacement policy that treats all cache entries equally therefore does not necessarily make efficient use of the available cache capacity. This motivates the development of size-aware replacement policies that incorporate the size of cache entries into their eviction decisions.

However, the cost associated with caching entries of different sizes depends on the characteristics of the storage environment. In an environment with low throughput, transferring a large column chunk requires more time than transferring a small one. Consequently, large cache entries are more expensive to retrieve and may deserve additional consideration when making eviction decisions. Conversely, in a high-latency environment, each individual remote access incurs a significant fixed cost. In this case, fetching several small entries can become expensive because accessing each entry separately may incur the full latency of a remote request. The relative importance of entry size therefore depends on whether transfer bandwidth or access latency dominates the cost of retrieving data.

As a result, there is no single size-aware policy that is necessarily optimal for all storage environments. The flexibility provided by the interchangeable replacement-policy interface is therefore also useful for incorporating different approaches to size awareness. Different policies can assign different importance to entry size or use different strategies for balancing entry size against access history. This makes it possible to evaluate which size-aware eviction strategies are most effective for a particular combination of workload, cache capacity, and remote storage characteristics.

5 Implementation

5.1 DuckDB

The implementation is based on DuckDB [RM19], a modern analytical database system designed primarily for Online Analytical Processing (OLAP) workloads. This makes DuckDB a suitable foundation for this thesis, as the investigated caching techniques and replacement policies target analytical query processing. OLAP workloads commonly process large amounts of data, perform scans over selected columns, and benefit from efficient access to data stored in remote storage systems.

An equally important reason for choosing DuckDB is its modular architecture and support for extensions. This made it possible to build upon the existing `duck-read-cache-fs` [den25] extension rather than implementing a complete caching mechanism from scratch. The extension provides a read cache that intercepts file reads and caches previously accessed data, thereby providing the basic functionality required for the experiments in this thesis.

The `duck-read-cache-fs` extension was particularly well suited as a foundation because of its comparatively small and comprehensive codebase. Its implementation focuses specifically on caching read requests and is sufficiently self-contained to allow the caching behavior to be modified without requiring extensive changes to unrelated parts of the database system. This made it possible to directly extend the existing cache implementation with the mechanisms required by the proposed design.

In particular, the extension provided a starting point for implementing the transition from fixed-size block caching to Parquet column chunk-based caching, supporting variable-sized cache entries, and integrating interchangeable cache replacement policies. Building on an existing read cache also allowed the implementation to retain the established path for handling file reads while concentrating the changes on the cache organization and eviction mechanisms investigated in this thesis.

Therefore, the combination of DuckDB and the `duck-read-cache-fs` extension provides both a representative OLAP database environment and a practical implementation basis for the proposed caching mechanisms. DuckDB supplies the analytical query engine and Parquet processing functionality, while the extension provides a compact and understandable foundation for implementing and evaluating the cache modifications.

5.2 Dynamic blocks

The implementation of column chunk-based caching extends the existing fixed-size block caching mechanism of the `duck-read-cache-fs` extension. The mechanism is optional and must first be explicitly enabled. This allows the existing caching behavior to remain unchanged unless column chunk-based caching is required. It can be enabled by setting the cache chunking mode as follows:

```
SET cache_httpfs_chunking='column_chunk';
```

Figure 5.1: Enabling column chunk-based caching.

Enabling this option does not automatically cause all accessed Parquet files to be cached using column chunks. Instead, individual Parquet files must be registered with the cache. This makes it possible to selectively use column chunk-based caching only for files for which the required layout information is available. A file is registered using the following function:

```
SELECT cache_httpfs_register_parquet_layout('/path/to/file.parquet');
```

Figure 5.2: Registering a Parquet file for column chunk-based caching.

During registration, the cache retrieves the physical layout of the Parquet file. Rather than implementing a separate Parquet metadata parser, the implementation leverages DuckDB's existing Parquet metadata functionality. The metadata is retrieved using a query of the following form:

```
SELECT {...}
FROM parquet_metadata('/path/to/file.parquet');
```

Figure 5.3: Retrieving the metadata required to determine the layout of a Parquet file.

From this metadata, the implementation extracts one pair for every column chunk consisting of its file offset and its size, represented as `(file_offset, chunk_size)`. These pairs describe the byte ranges occupied by the column chunks within the Parquet file. The resulting list of column chunk ranges is then stored in an unordered map, using the file path as the key and the corresponding list of column chunk offsets and sizes as the value. Consequently, the cache can efficiently determine whether a read request for a particular file corresponds to one or more registered column chunks.

After a file has been registered, subsequent read requests issued by the DBMS are handled by the column chunk caching mechanism. Each request specifies a memory

section of the file that is required by the DBMS. This requested byte range is passed to the column chunk planner, which translates the request into a list of column chunks required to satisfy it. If the requested range lies completely within a single column chunk, the resulting list contains only that column chunk. If the request spans multiple column chunks, the planner returns multiple entries, corresponding to all column chunks intersected by the requested byte range.

Each column chunk in the resulting list is subsequently processed independently. The cache first checks whether the complete column chunk is already present in the cache. On a cache hit, the required bytes are copied directly from the cached column chunk to the destination buffer requested by the DBMS. On a cache miss, the complete column chunk must be fetched from persistent storage and inserted into the cache before the requested data can be copied to the destination.

Since column chunks can have different sizes, inserting a missed column chunk may require freeing a variable amount of cache capacity. If sufficient capacity is not available, one or more existing cache entries have to be evicted before the new column chunk can be inserted. The decision of which entries should be evicted is delegated to the cache replacement policy. The implementation of this mechanism, including the handling of variable-sized cache entries and the replacement policies used for eviction decisions, is described in the following section.

Column chunk-based caching does not replace the existing fixed-size block caching mechanism entirely. Files that have not been registered continue to be handled using fixed-size cache blocks. Furthermore, not every byte range within a registered Parquet file belongs to a column chunk. For example, Parquet metadata and other file regions outside registered column chunks cannot be mapped by the column chunk planner. These requests are therefore also handled by the original fixed-size caching mechanism.

The size of these fixed-size cache blocks can be configured using the existing cache configuration option:

```
SET cache_httpfs_cache_block_size={block_size};
```

Figure 5.4: Configuring the block size used by the fallback fixed-size caching mechanism.

This hybrid approach preserves the ability of the cache to handle arbitrary read requests while using the Parquet column chunk structure whenever it is available. Registered column chunks are cached as complete, variable-sized entries, whereas unregistered files and byte ranges that cannot be associated with a column chunk continue to use the established fixed-size block caching mechanism.

5.3 Replacement policies

5.3.1 General Replacement Policy Interface

To support interchangeable cache replacement policies, the implementation defines a common abstract interface that separates replacement decisions from the general cache management logic. The cache itself is responsible for storing entries, tracking the available capacity, and performing reads and evictions, while the replacement policy maintains the information required to determine which entries should be removed.

The interface is defined by the `EvictionPolicy` abstract base class shown in Figure 5.5. Every replacement policy implements the same set of methods and can consequently be used by the cache without requiring policy-specific changes to the cache implementation.

The interface is based on methods that notify the replacement policy about relevant changes to the state of the cache. When a cached entry is accessed, the cache calls `OnHit`. The policy can use this information to update its internal state according to its replacement strategy. For example, policies that consider recency can update the position of the accessed entry, while policies that do not depend on access order can ignore the notification.

When a new entry is inserted into the cache, `OnInsert` is called. In addition to the identifier of the cache entry, the method receives the size of the entry in bytes and the cost of fetching it from persistent storage. Providing these values through the common interface is particularly important because the column chunk-based caching mechanism uses variable-sized cache entries. A replacement policy can therefore take the amount of cache capacity occupied by an entry into account when making eviction decisions. Similarly, the retrieval cost can be used by policies that balance the cost of evicting an entry against the cache capacity that can be freed. Policies that do not require either value can simply ignore them. Consequently, size-aware and cost-aware policies can be implemented without requiring changes to the interface or to the cache code that invokes it.

The `OnErase` method is called when an entry leaves the cache for a reason other than eviction. This includes explicit invalidation, timeouts, or changes to the cache block mapping. The notification allows the replacement policy to remove the corresponding entry from its internal state and ensures that the state maintained by the policy remains consistent with the actual contents of the cache.

When the cache requires additional capacity, it requests an eviction decision through `PopVictim`. The replacement policy selects the entry that should be removed next and returns it through the `victim` parameter. The method returns `false` when the policy does not contain an entry that can be evicted. In this case, the cache treats the result as

```
class EvictionPolicy {
public:
        virtual ~EvictionPolicy() = default;

        // A cached entry was read.
        virtual void OnHit(const InMemCacheBlock &key) = 0;

        // An entry was admitted.
        virtual void OnInsert(const InMemCacheBlock &key,
                              idx_t size,
                              double cost_us) = 0;

        // An entry left the cache for a reason other than eviction.
        virtual void OnErase(const InMemCacheBlock &key) = 0;

        // Choose the next entry to evict.
        virtual bool PopVictim(InMemCacheBlock &victim) = 0;

        // Admission control.
        virtual bool Admit(idx_t size, double cost_us) {
                return true;
        }

        virtual void Clear() = 0;

        virtual string GetName() const = 0;
};
```

Figure 5.5: The abstract interface implemented by all cache replacement policies.

an indication that no additional capacity can be freed by the policy.

In addition to eviction decisions, the interface provides support for admission control through the `Admit` method. Before a newly fetched entry is inserted into the cache, the cache can ask the replacement policy whether the entry should be admitted. The default implementation always returns `true`, meaning that all entries are admitted by default. Policies can override this method to reject entries that are unlikely to benefit from caching. This mechanism is useful for policies designed to avoid cache pollution caused by scan-heavy workloads, where data may only be accessed once and caching it could displace entries with a higher probability of reuse.

The `Clear` method removes all state maintained by the replacement policy. This allows the cache to reset the policy when the cache itself is cleared. Finally, `GetName` returns an identifier for the policy, which can be used for configuration, logging, and the evaluation of different replacement strategies.

By defining all interactions between the cache and the replacement policy through this common interface, the cache implementation remains independent of the specific replacement strategy being used. Conventional policies, size-aware policies, and policies with admission control can all implement the same interface and can therefore be exchanged without modifying the general cache implementation. This provides the flexibility required to evaluate different replacement strategies for different workloads, cache sizes, and storage environments.

5.3.2 LRU Policy

The LRU policy was implemented as a reference replacement policy. Since LRU corresponds to the eviction behavior of the original in-memory cache, it provides a baseline for experiments that change the cache granularity while keeping the replacement behavior unchanged.

The policy maintains the access order of cached entries using a `std::list`. The most recently accessed entry is stored at the front of the list, while the least recently accessed entry is located at the back. To allow entries to be located efficiently, the implementation additionally uses a `std::unordered_map` that maps each `InMemCacheBlock` to its corresponding iterator in the list.

On a cache hit, the accessed entry is moved to the front of the list. Newly inserted entries are likewise placed at the front. When the cache requires an eviction, the entry at the back of the list is selected as the victim, corresponding to the least recently used cached entry.

The policy also implements the `OnErase` callback to remove entries that leave the cache for reasons other than eviction. Together with the `Clear` method, this ensures that the state maintained by the replacement policy remains consistent with the contents of

the cache.

The LRU implementation does not use the entry size or retrieval cost provided through the replacement policy interface. It therefore treats all cache entries equally when determining their eviction order. This makes it suitable as a reference policy for comparing the effects of column chunk-based caching independently of size-aware replacement decisions.

5.3.3 Size-Aware LRU Policy

The size-aware LRU policy extends the LRU policy described above by additionally considering the size of cache entries during eviction. It is primarily implemented for benchmarking purposes to evaluate whether incorporating entry size into eviction decisions can improve cache performance compared to conventional LRU.

The policy maintains the same LRU ordering as the standard implementation. Cache hits and insertions move entries to the front of the `std::list`, while eviction candidates are considered from the back. In addition to the cache block identifier, each entry maintains a counter that records how many times it has reached the eviction position.

When an entry reaches the end of the LRU list, the policy determines whether it should be evicted immediately or receive another round through the list. The number of rounds is calculated based on the entry size. For a block of size s, the number of rounds is calculated as

$$T_{\text{rounds}}(s) = \min\left(\left\lfloor \frac{s_{\text{ref}}}{s} \right\rfloor, T_{\max}\right), \tag{5.1}$$

where s_{ref} denotes the reference block size and $T_{\max}$ denotes the maximum number of rounds. In the implementation, s_{ref} is set to 1 MiB and $T_{\max}$ is set to 10. Consequently, entries smaller than the reference block size can receive multiple additional rounds, whereas entries with a size larger than the reference size receive no additional rounds. Smaller cache entries are therefore retained for longer before being evicted. The chosen reference block size of 1 MiB results in the majority of column chunks in TPC-H SF100 receiving more than one round in the list, as shown in Figure 5.6.

If an entry has not yet used all of its allowed rounds when it reaches the eviction position, its counter is incremented and the entry is moved back to the front of the LRU list. It will subsequently move towards the eviction position again as other entries are accessed. Once its counter reaches the calculated maximum number of rounds, the entry is evicted when it next reaches the end of the list. A cache hit resets the counter, giving the entry a fresh set of possible rounds.

This mechanism preserves the basic recency-based behavior of LRU while giving smaller entries additional protection from eviction. The policy therefore provides a

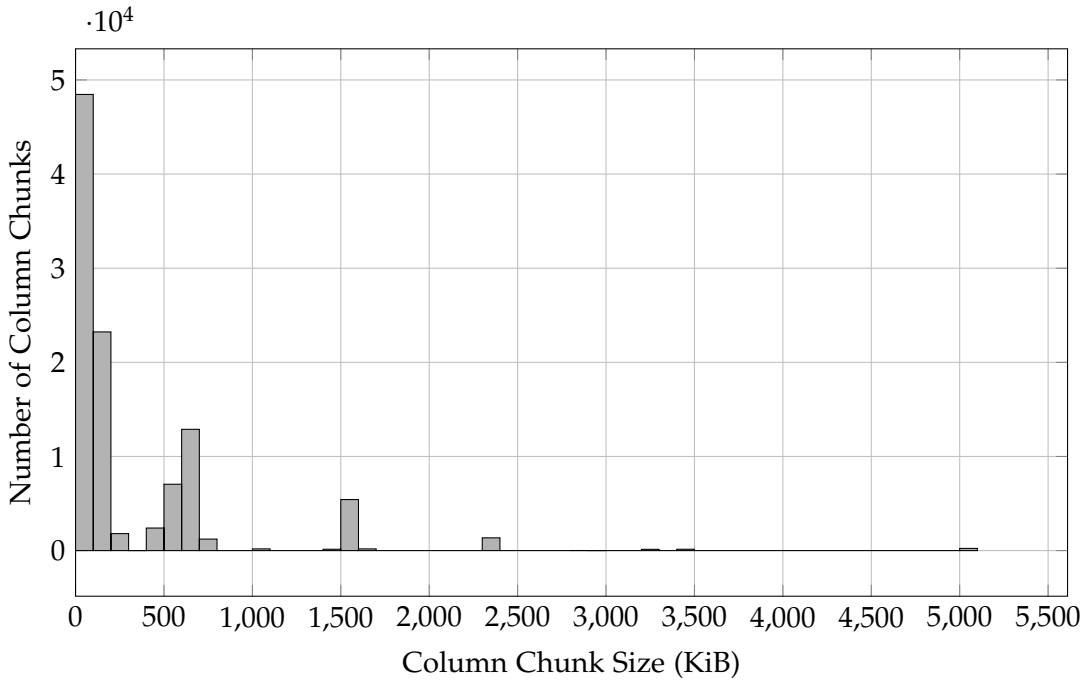


Figure 5.6: Distribution of column chunk sizes in TPC-H SF100 using 100 KiB bins.

simple mechanism for investigating whether considering the amount of cache capacity occupied by an entry can improve performance. Its primary purpose is not to propose a universally optimal size-aware replacement strategy, but to serve as a benchmark for determining whether size-aware eviction decisions provide an advantage for the evaluated workloads and cache configurations.

5.3.4 S3-FIFO Replacement Policy

S3-FIFO is a cache replacement algorithm proposed by Yang et al. [Yan+23]. The algorithm was designed as a simple and scalable alternative to LRU-based replacement policies. Its central observation is that many cache workloads contain a large number of objects that are accessed only once within a short time window. Keeping such objects in the cache can displace objects that are accessed repeatedly. S3-FIFO addresses this problem by quickly filtering out these "one-hit wonders" instead of allowing them to occupy a large portion of the cache. The authors show across a large collection of real-world cache traces that S3-FIFO can achieve lower miss ratios than several established replacement policies while also providing better scalability than LRU-based approaches [Yan+23].

S3-FIFO uses three First In, First Out (FIFO) queues: a small queue *S*, a main queue *M*, and a ghost queue *G*. The small queue acts as a probationary area for newly inserted objects. An object initially enters *S* and is only promoted to the main queue if it is accessed again before reaching the eviction position. Objects that are not reused are therefore removed quickly without occupying space in the main cache. The main queue contains objects that have demonstrated reuse and consequently receive longer protection from eviction. The ghost queue does not contain cached data. Instead, it stores identifiers of objects that were recently evicted from the small queue. If an object is requested again while its identifier is still present in the ghost queue, it is considered to have demonstrated reuse and can be inserted directly into the main queue. This mechanism allows S3-FIFO to identify objects that were evicted from the probationary queue too early.

The small queue occupies approximately 10% of the cache capacity, while the main queue uses the remaining 90%. The ghost queue has a capacity based on the number of entries in the main queue and stores only metadata. Unlike LRU, accesses do not change the position of an object in the queue. Instead, S3-FIFO maintains a small capped frequency counter for each cached object. This counter records whether an object has been accessed since its position in the FIFO queue was established. During eviction, this information is used to determine whether an object should be promoted, given another pass through the main queue, or removed.

This design addresses two important limitations of LRU. First, LRU retains recently accessed objects even when they are unlikely to be accessed again. In a workload containing many sequential or one-time accesses, such objects can push frequently reused objects out of the cache. The small FIFO queue of S3-FIFO provides a form of quick demotion that filters these objects before they enter the main cache. Second, LRU requires the ordering of the cache to be modified on every cache hit because the accessed object has to be moved to the most-recently-used position. S3-FIFO does not modify queue order on a hit and only updates a small frequency counter. This reduces the amount of metadata manipulation required on the hit path, which can be beneficial in highly concurrent workloads. The evaluation of the original S3-FIFO work reports both improved miss ratios and substantially higher throughput compared with LRU-based implementations [Yan+23].

In this work, the S3-FIFO policy is implemented as an additional implementation of the common `EvictionPolicy` interface described in the previous section. The implementation uses three FIFO queues represented by `std::list`. The small and main queues contain the actual cached blocks, while the ghost queue contains only `InMemCacheBlock` identifiers. An additional hash table provides constant-time lookup of blocks and stores the metadata associated with each cached entry.

The capacity of the small and main queues is determined when the policy is con-

structed. The small queue is assigned 10% of the configured cache capacity, while the main queue receives the remaining 90%:

```
small_capacity = cache_size / 10
main_capacity  = cache_size - small_capacity
```

Since cache entries have variable sizes, the implementation tracks the total size of the entries currently contained in each queue. Consequently, the queue capacities are expressed in bytes rather than simply as a number of entries. The small queue and main queue maintain separate counters, `small_size` and `main_size`, which are updated whenever an entry is inserted, promoted, or removed.

Each cached block has an associated queue, size, frequency counter, and iterator identifying its position in the corresponding FIFO list. The frequency counter is capped at three. On a cache hit, the implementation therefore only increments this counter if it has not already reached the maximum value. In contrast to LRU, the entry is not moved within the FIFO queue. This keeps the hit operation independent of the queue ordering.

When a new block is inserted, the policy first checks whether the block is already present in the cache. Duplicate insertions are ignored. If the block's identifier is found in the ghost queue, the block is inserted directly into the main queue instead of the small queue. Its previous ghost entry is removed and its frequency counter is initialized to zero. This implements the mechanism by which blocks that have recently been evicted from the small queue can bypass the probationary stage after demonstrating reuse. If the block is not present in the ghost queue, it is inserted at the front of the small queue with a frequency of zero.

Eviction from the small queue is performed by examining the oldest entry at the back of the FIFO queue. If the entry has been accessed since insertion, indicated by a frequency greater than zero, it is promoted to the main queue instead of being evicted. The entry is removed from the back of the small queue and inserted at the front of the main queue. If the oldest entry has not been accessed, it is considered a one-hit wonder and is evicted. Before being removed completely, its identifier is added to the ghost queue.

The ghost queue therefore provides information about recently evicted blocks without retaining their data. When an entry is added to the ghost queue, its identifier is inserted at the front. The implementation limits the number of ghost entries to the current number of entries in the main queue. When the ghost queue exceeds this capacity, its oldest entries are removed. This allows recently evicted blocks to be recognized while bounding the amount of metadata maintained by the policy.

The main queue uses a FIFO-reinsertion mechanism. When its oldest entry is considered for eviction, its frequency counter determines whether it should receive

another pass through the queue. If the counter is greater than zero, it is decremented by one and the entry is moved from the back to the front of the main queue. The entry therefore receives another opportunity to remain in the cache. If the counter has reached zero, the entry is removed and selected as the eviction victim. Consequently, frequently accessed entries can survive multiple passes through the main queue, while entries that have not been accessed since their previous pass are eventually evicted.

The `PopVictim` operation first ensures that the small queue does not exceed its assigned capacity. If it does, entries are processed from the small queue until either an entry can be evicted or an accessed entry is promoted to the main queue. The main queue is then processed if it exceeds its assigned capacity. If neither queue exceeds its target capacity, the small queue is preferred as the source of the next eviction candidate. This preserves its role as the probationary portion of the cache. If the small queue is empty, an entry is instead selected from the main queue.

The resulting implementation therefore retains the central properties of S3-FIFO while integrating it into the variable-sized cache used in this thesis. In particular, the 10%/90% partition is applied to the amount of cached data rather than to a fixed number of blocks, and the frequency counters and queue transitions operate independently of the size of an individual block. The size-aware extensions investigated later build on this implementation by modifying how variable-sized blocks are treated during replacement.

5.3.5 Size-Aware S3-FIFO Replacement Policies

The standard S3-FIFO policy does not explicitly consider the size of a cache entry when making replacement decisions. This is particularly relevant for the column chunk-based cache used in this thesis, since Parquet column chunks can have substantially different sizes. Treating all blocks equally can therefore result in a large block having the same replacement priority as a much smaller block, even though retaining the smaller block requires comparatively little cache space and can potentially prevent an additional read.

To investigate whether the benefits of S3-FIFO can be further improved by considering the size of cache entries, three size-aware variants were implemented. All variants retain the basic structure of S3-FIFO with a small probationary queue, a main queue, and a ghost queue. The variants differ in where block size influences the replacement behavior. The first variant makes promotion from the small queue size-aware, the second gives smaller blocks additional rounds through the main queue, and the third combines both approaches.

S3-FIFO Promotion

The first variant, referred to as `s3fifo_promotion`, takes block size into account when promoting entries from the small queue to the main queue. In standard S3-FIFO, an entry that is accessed at least once while residing in the small queue is promoted to the main queue. This treats a small and a large block equally, even though promoting a large block consumes substantially more cache capacity.

The size-aware variant therefore requires larger blocks to demonstrate stronger evidence of reuse before they are promoted. The required number of hits is determined by the block size relative to a configurable reference size:

$$T_{\text{promotion}}(s) = \min\left(\max\left(1, \left\lceil \frac{s}{s_{\text{ref}}} \right\rceil\right), T_{\max}\right), \tag{5.2}$$

where s is the size of the cache block, s_{ref} is the reference block size, and $T_{\max}$ is the maximum promotion threshold.

Consequently, a block up to the reference size requires one hit for promotion. A block approximately twice the reference size requires two hits, while blocks at least $T_{\max}$ times the reference size require at most $T_{\max}$ hits. The threshold is bounded to prevent very large blocks from requiring an impractically high number of accesses.

The frequency counter of an entry is incremented on every cache hit while the entry resides in the cache, up to the maximum promotion threshold. When the entry reaches the eviction position of the small queue, its frequency is compared with the size-dependent promotion threshold. If the threshold has been reached, the entry is promoted to the main queue. Otherwise, the block is evicted and its identifier is added to the ghost queue.

This modification favors small blocks during admission to the main queue. A large block must demonstrate more reuse before it is allowed to occupy the long-lived portion of the cache. The approach therefore attempts to balance the benefit of retaining a frequently reused block against the amount of cache capacity consumed by that block.

The remainder of the S3-FIFO behavior is unchanged. In particular, the main queue still uses the standard frequency-based second-chance mechanism, and a block that is accessed after being evicted from the small queue can still be admitted directly to the main queue through a ghost hit.

S3-FIFO Rounds

The second variant, referred to as `s3fifo_rounds`, instead considers block size when entries are processed in the main queue. The motivation is that a small block and a large block consume different amounts of cache capacity, but the standard S3-FIFO policy gives both the same number of opportunities to remain in the main queue.

In this variant, each block is assigned a size-dependent eviction cost. The cost is calculated from the block size relative to a configurable reference size:

$$C_{\text{eviction}}(s) = \min\left(\max\left(1, \left\lceil \frac{s}{s_{\text{ref}}} \right\rceil\right), C_{\max}\right), \tag{5.3}$$

where s is the block size, s_{ref} is the reference size, and $C_{\max}$ limits the maximum cost.

When the oldest entry in the main queue is processed, its frequency counter represents the number of remaining opportunities it has to avoid eviction. Instead of simply decrementing this counter by one as in standard S3-FIFO, the size-aware policy decrements it by the block's eviction cost. If sufficient frequency remains, the block is moved back to the front of the main queue and receives another opportunity to remain cached. Otherwise, it is evicted.

As a result, smaller blocks require fewer frequency credits for each pass through the main queue and can therefore survive more passes than larger blocks. For example, assuming a reference size of 1 MiB, a block smaller than or equal to 1 MiB has an eviction cost of one, whereas a block of approximately 3 MiB has an eviction cost of three. A small block can consequently receive several passes through the main queue while consuming relatively little cache capacity, whereas a large block is removed more quickly unless it is accessed sufficiently often.

The small queue remains size-independent in this variant. Newly inserted blocks are promoted after the standard S3-FIFO promotion condition has been met. Size is only considered once an entry reaches the eviction position of the main queue. This isolates the effect of giving smaller blocks additional rounds in the main queue and allows it to be evaluated independently from size-aware promotion.

S3-FIFO Both

The third variant, referred to as `s3fifo_both`, combines the two previously described approaches. It uses a size-dependent promotion threshold when processing the small queue and a size-dependent eviction cost when processing the main queue.

A newly inserted block therefore first has to demonstrate sufficient reuse relative to its size before it is promoted to the main queue. Once promoted, its size also influences how quickly its frequency credits are consumed when it reaches the eviction position of the main queue. Small blocks can consequently benefit from both a lower promotion threshold and a lower eviction cost, while larger blocks require stronger evidence of reuse and consume their frequency credits more quickly.

The combined variant is intended to investigate whether the two size-aware mechanisms complement each other. Size-aware promotion controls which blocks are admitted to the long-lived main queue, while size-aware rounds determine how long

admitted blocks remain there. This provides two separate points at which block size can influence the replacement decision. The evaluation in the following chapter compares the individual variants with standard S3-FIFO to determine whether these modifications improve cache performance for the variable-sized Parquet column chunks used in this thesis.

6 Evaluation

6.1 Evaluation Setup

The benchmarks were conducted on a server running NixOS 26.05 (Yarara) with a Linux 7.1.9 kernel on an x86_64 architecture. The system is equipped with an AMD EPYC 9654P processor providing 96 physical cores and 192 logical Central Processing Units (CPUs). The machine has approximately 770 GiB of main memory and no configured swap space.

The TPC-H Parquet files used for the benchmarks are stored on a Samsung PM9A3 Non-Volatile Memory Express (NVMe) Solid-State Drive (SSD).

Table 6.1 summarizes the relevant characteristics of the evaluation environment.

Table 6.1: Hardware and software configuration used for the evaluation.

Component	**Configuration**
Operating system	NixOS 26.05 (Yarara)
Kernel	Linux 7.1.9
CPU	AMD EPYC 9654P
Physical cores	96
Logical CPUs	192
Main memory	770 GiB
Swap	None
Local storage	3.5 TB NVMe SSD

The local NVMe storage provides the persistent storage layer used for the benchmarks. Remote storage latency is simulated as described in the following section, allowing the experiments to reproduce remote storage characteristics while keeping the benchmark data on local storage.

6.2 Simulation of Remote Storage Latency

Since the benchmarks are intended to evaluate a cache in a cloud-native environment, accessing data from persistent storage should incur a latency representative of remote

storage. Instead of relying on an actual remote object store, the latency is simulated locally. This provides greater flexibility for the evaluation, as different latency and bandwidth characteristics can be tested without changing the underlying storage system or depending on network conditions.

The simulated remote read consists of two components: a fixed latency and a bandwidth-dependent transfer time. For each cache miss, the size of the requested block is used to calculate the transfer time based on the configured bandwidth. The fixed latency is then added to this value, resulting in the total simulated read time.

The simulation accounts for the time already required to read the data from the local storage. Before the local read, the start time of the operation is recorded. After the read has completed, the elapsed time is compared with the simulated remote read time. Only the remaining difference is added as a delay. Consequently, the simulation does not unnecessarily add remote latency on top of an already sufficiently long local read. The resulting delay is applied by suspending the executing thread for the remaining duration.

6.3 Benchmark Parameters

The TPC-H benchmark was selected for the evaluation as a well-established benchmark for analytical database systems. It consists of a set of decision support queries that access data in different ways and therefore provides a representative workload for evaluating cache behavior in an analytical query engine. Query Q15 was excluded from the evaluation due to technical difficulties that could not be resolved within the available time. The benchmarks therefore cover 21 of the 22 TPC-H queries.

A scale factor of 100 was chosen for the TPC-H dataset, resulting in approximately 35 GiB of data. This scale factor provides a compromise between using a sufficiently large dataset to represent a realistic analytical workload and keeping the overall benchmark execution time within reasonable limits.

The queries were executed using 64 threads. The relatively high degree of parallelism was chosen to reduce the influence of DBMS-internal processing on the measured execution times. With a sufficient number of threads, multiple data requests can be processed concurrently, making the performance of the cache in delivering the required data to the query engine a more significant factor in the overall execution time.

Cold execution time is measured with an initially empty cache. Warm execution time is measured after executing the same workload once beforehand, where a workload consists of either an individual query or a query sequence. The cache is not cleared between this initial execution and the measured execution, so the warm measurement starts with the cache contents left by the preceding execution. A warm cache therefore

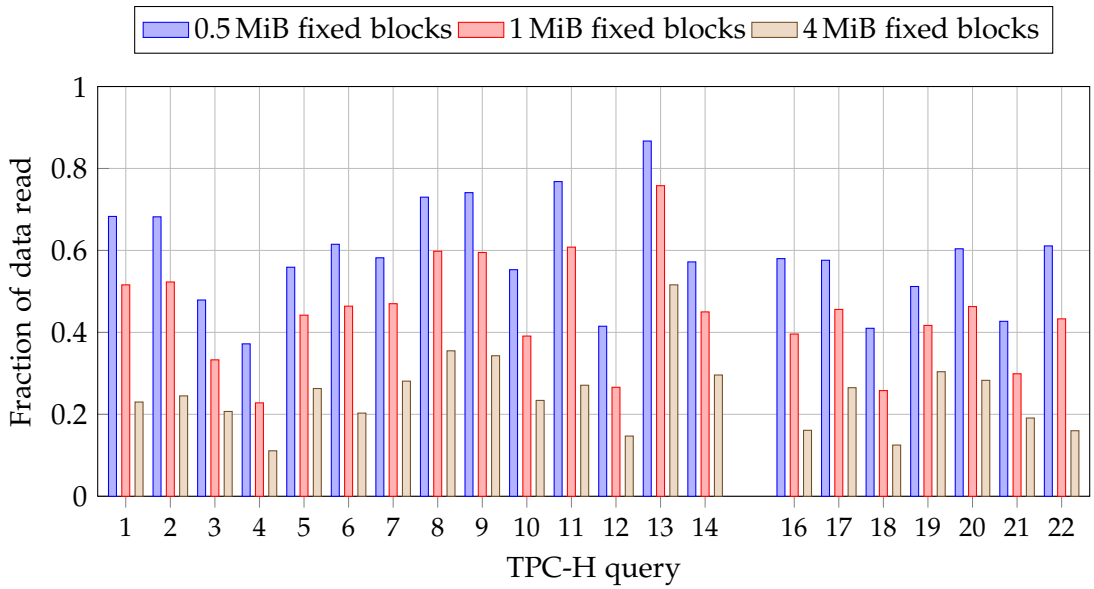


Figure 6.1: Ratio of bytes read using column chunk-based caching to bytes read using fixed-size caching. Each group represents one TPC-H query. Lower values indicate a larger reduction in transferred data through column chunk-based caching.

does not necessarily contain the complete working set, as its contents depend on the available cache capacity and the replacement policy.

Each reported execution time represents a single measured execution of the corresponding workload.

For the simulation of a remote S3 [Ama26] object store, a latency of 20 ms and a transfer rate of approximately 0.5 Gbit/s were used. These values were obtained through measurements in a realistic environment and were chosen to provide representative characteristics of remote object storage. Using these parameters for the simulation allows the experiments to reproduce the relevant latency and bandwidth characteristics while retaining the flexibility of running the benchmarks against local storage.

6.4 Dynamic block size

6.4.1 Dynamic Block Sizes on Initial Load

Figure 6.2 compares the execution time of the TPC-H queries when using fixed-size cache blocks and dynamically sized blocks based on Parquet column chunks. The

measurements were performed with a cache large enough to hold the complete dataset of the respective workload. Consequently, the measured cold execution time primarily reflects the cost of retrieving the required data from persistent storage rather than cache evictions during query execution.

The results show that neither caching strategy consistently outperforms the other. For some queries, fixed-size blocks result in a lower execution time, while for others, column chunk-based caching is faster. This difference can be explained by two opposing effects. Fixed-size blocks can benefit from implicitly prefetching data that is located close to the requested data. This can reduce the number of individual read operations, but may also result in more data being read than is actually required by the query. Column chunk-based caching, on the other hand, allows the cache to retrieve data more precisely and can therefore reduce the total amount of data transferred. This difference can be significant, as can be seen in Figure 6.1. However, because column chunks are individually addressed, accessing the same data can require a larger number of read operations.

The difference between these effects can be illustrated using queries Q01 and Q21. For Q01, considering the first row group, the fixed-size configuration requires three reads corresponding to blocks 2, 3, and 4. These blocks contain a total of 1536 KiB of data, as shown in Figures 6.3 and 6.4. With column chunk-based caching, only approximately 1069 KiB need to be read. However, the required column chunks are distributed over seven individual reads. Since the required column chunks are located relatively close to each other, the fixed-size blocks implicitly prefetch additional data while requiring fewer read operations. In this case, the reduction in the number of reads outweighs the additional data transferred, resulting in a lower execution time for fixed-size blocks.

The opposite behavior can be observed for Q21. For the first row group, fixed-size caching requires three reads of blocks 0, 1, and 4, resulting in 1536 KiB of data being read, as shown in Figures 6.5 and 6.6. Column chunk-based caching requires four reads but transfers only approximately 817 KiB. In this case, the required column chunks are more widely distributed, meaning that the additional data fetched by the fixed-size blocks provides less useful prefetching. This example illustrates the reduction in transferred data, but does not by itself explain the overall execution-time improvement observed for Q21, which also depends on the complete query access pattern and concurrent execution of read requests.

These examples demonstrate that the benefit of dynamic block sizes depends on the physical layout of the required data and the relationship between read latency and transfer cost. When relevant data is located close together, fixed-size blocks can exploit this locality through implicit prefetching and reduce the number of storage accesses. When relevant data is more widely distributed, column chunk-based caching can avoid

unnecessary data transfers by reading only the required chunks. Thus, dynamic block sizes do not inherently provide better performance when initially fetching data into the cache, but they can still improve cache performance in other ways, as will be shown in the following section.

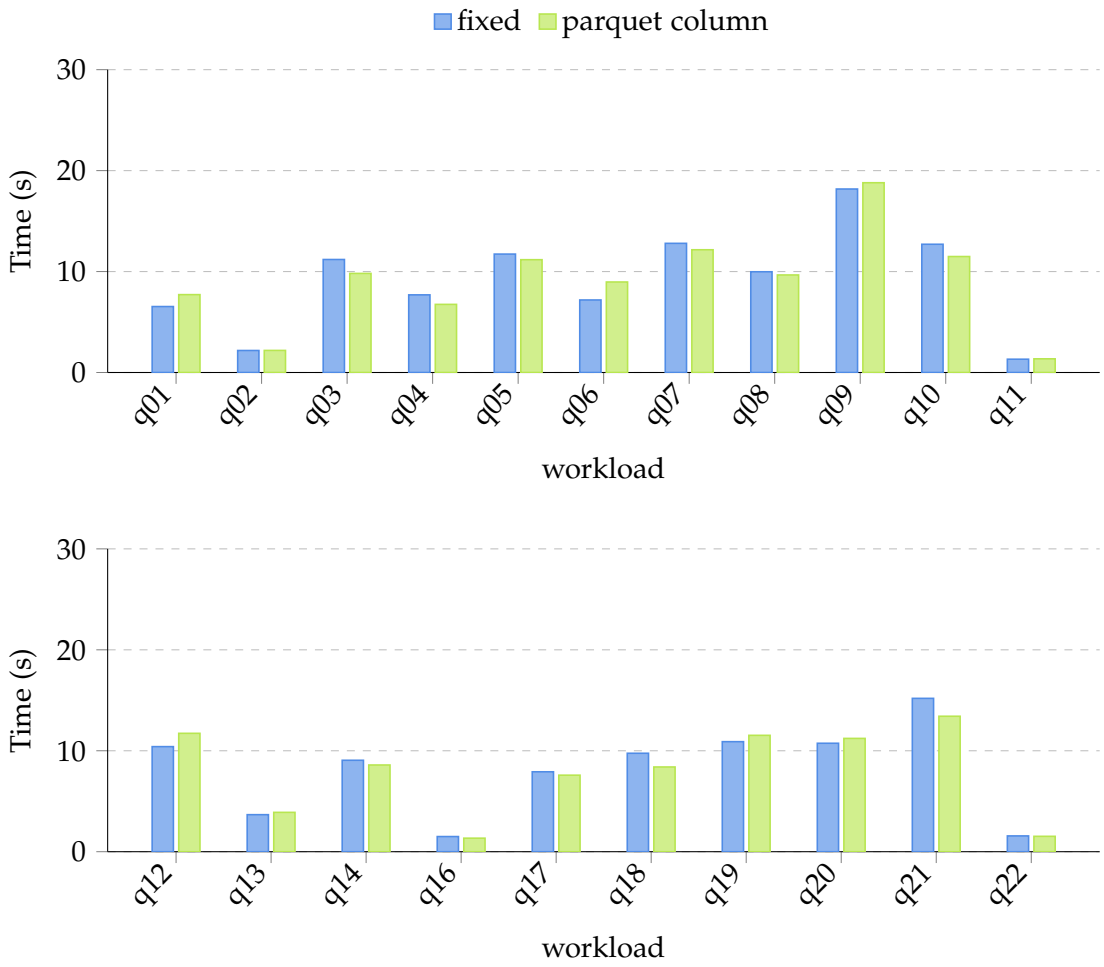


Figure 6.2: Cold execution time for parquet column and fixed-size caching with 64 threads, a block size of 512 KiB for fixed blocks, LRU replacement, a cache size of 100% of the dataset, and the TPC-H SF100 benchmark. Simulated storage has a bandwidth of 0.5 Gbit/s and a latency of 20 ms.

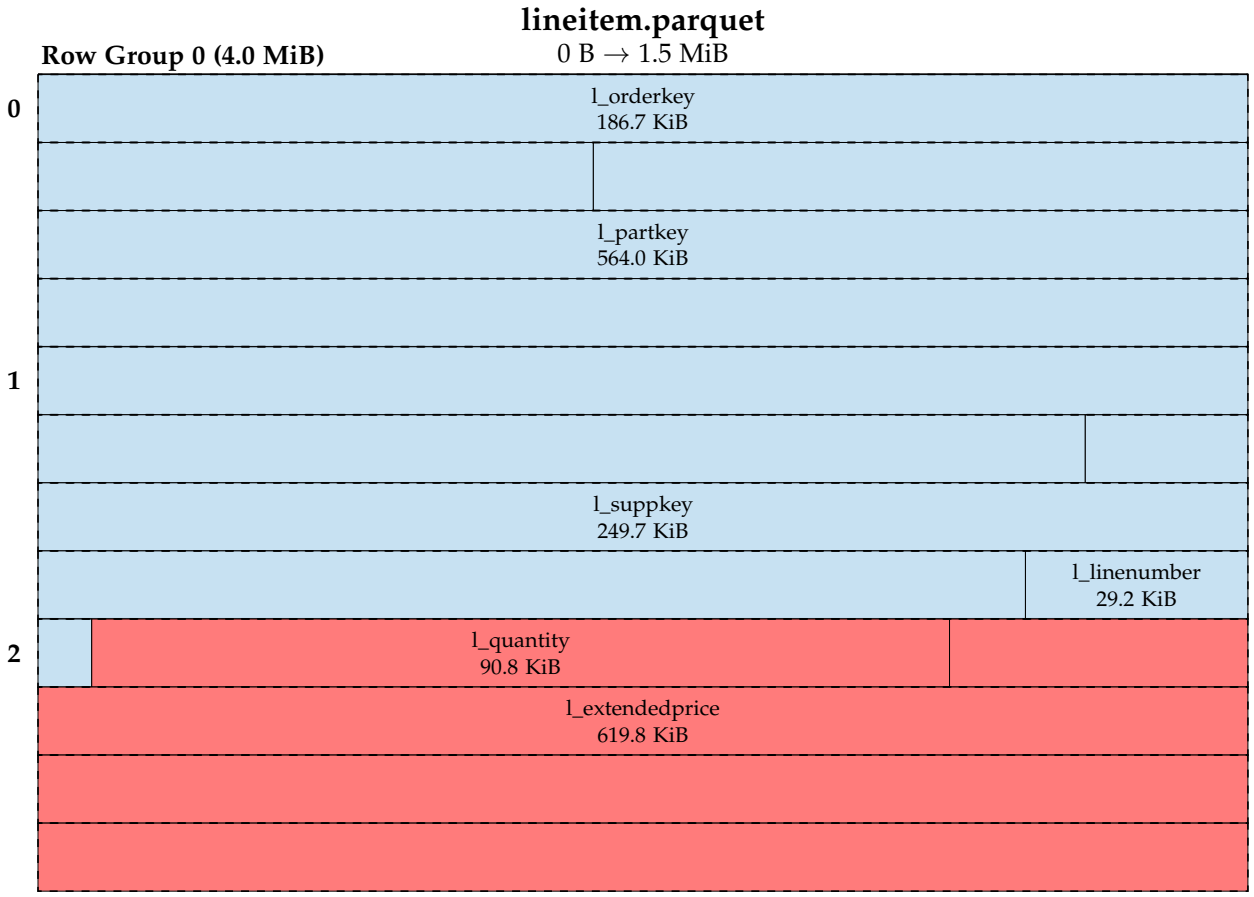


Figure 6.3: Physical layout of `lineitem.parquet` for the byte range from 0 B to 1.5 MiB, showing row group 0. Column chunks accessed by TPC-H Q01 ($l_{\text{returnflag}}$, $l_{\text{linestatus}}$, l_{quantity}, $l_{\text{extendedprice}}$, l_{discount}, l_{tax}, and l_{shipdate}) are shown in red, while all other column chunks are shown in light blue. Each line represents 128 KiB and every four lines form a 0.5 MiB address-space block.

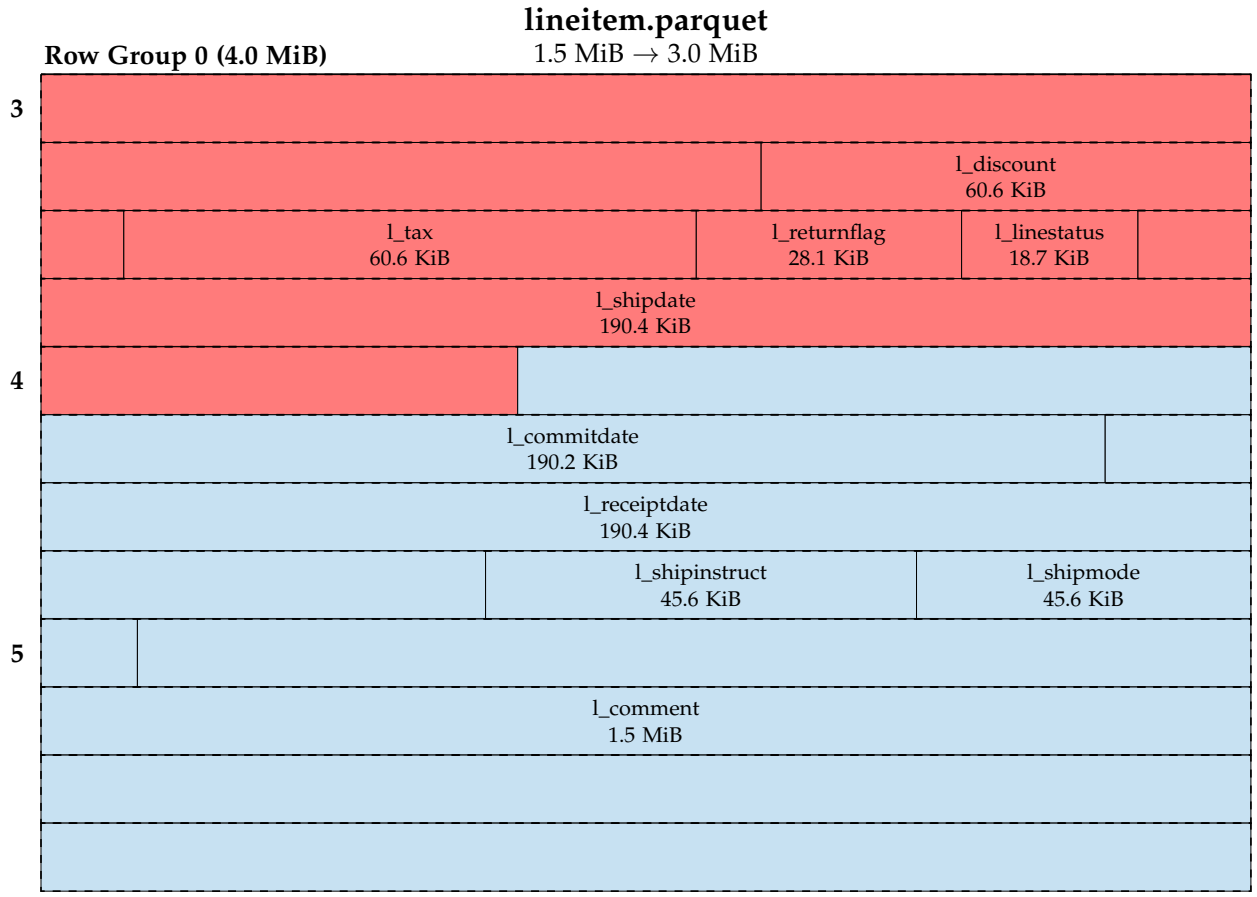


Figure 6.4: Physical layout of `lineitem.parquet` for the byte range from 1.5 MiB to 3.0 MiB, showing row group 0. Column chunks accessed by TPC-H Q01 ($l_{\text{returnflag}}$, $l_{\text{linestatus}}$, l_{quantity}, $l_{\text{extendedprice}}$, l_{discount}, l_{tax}, and l_{shipdate}) are shown in red, while all other column chunks are shown in light blue. Each line represents 128 KiB and every four lines form a 0.5 MiB address-space block.

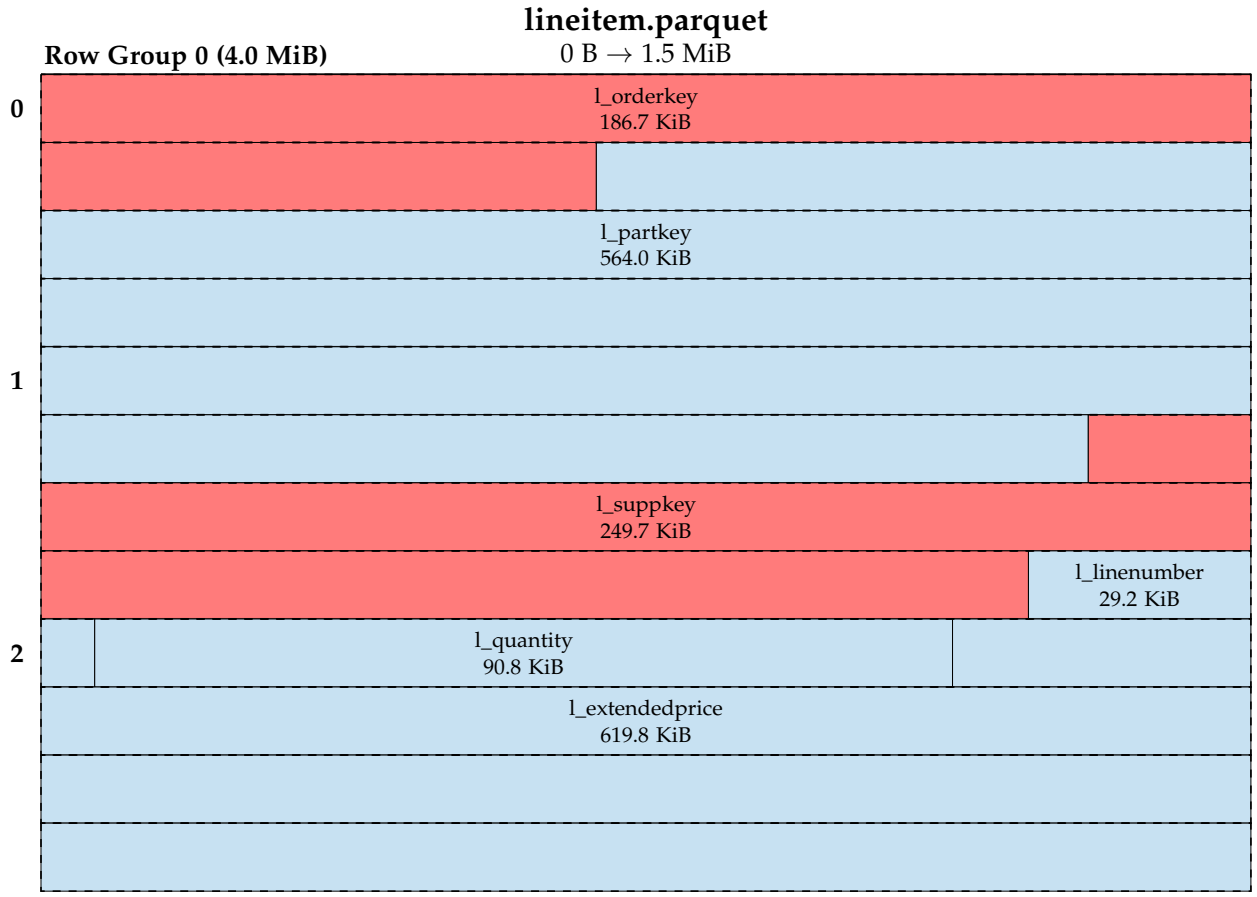


Figure 6.5: Physical layout of `lineitem.parquet` for the byte range from 0 B to 1.5 MiB, showing row group 0. Column chunks accessed by TPC-H Q21 (l_{suppkey}, l_{orderkey}, $l_{\text{receiptdate}}$, and $l_{\text{commitdate}}$) are shown in red, while all other column chunks are shown in light blue. Each line represents 128 KiB and every four lines form a 0.5 MiB address-space block.

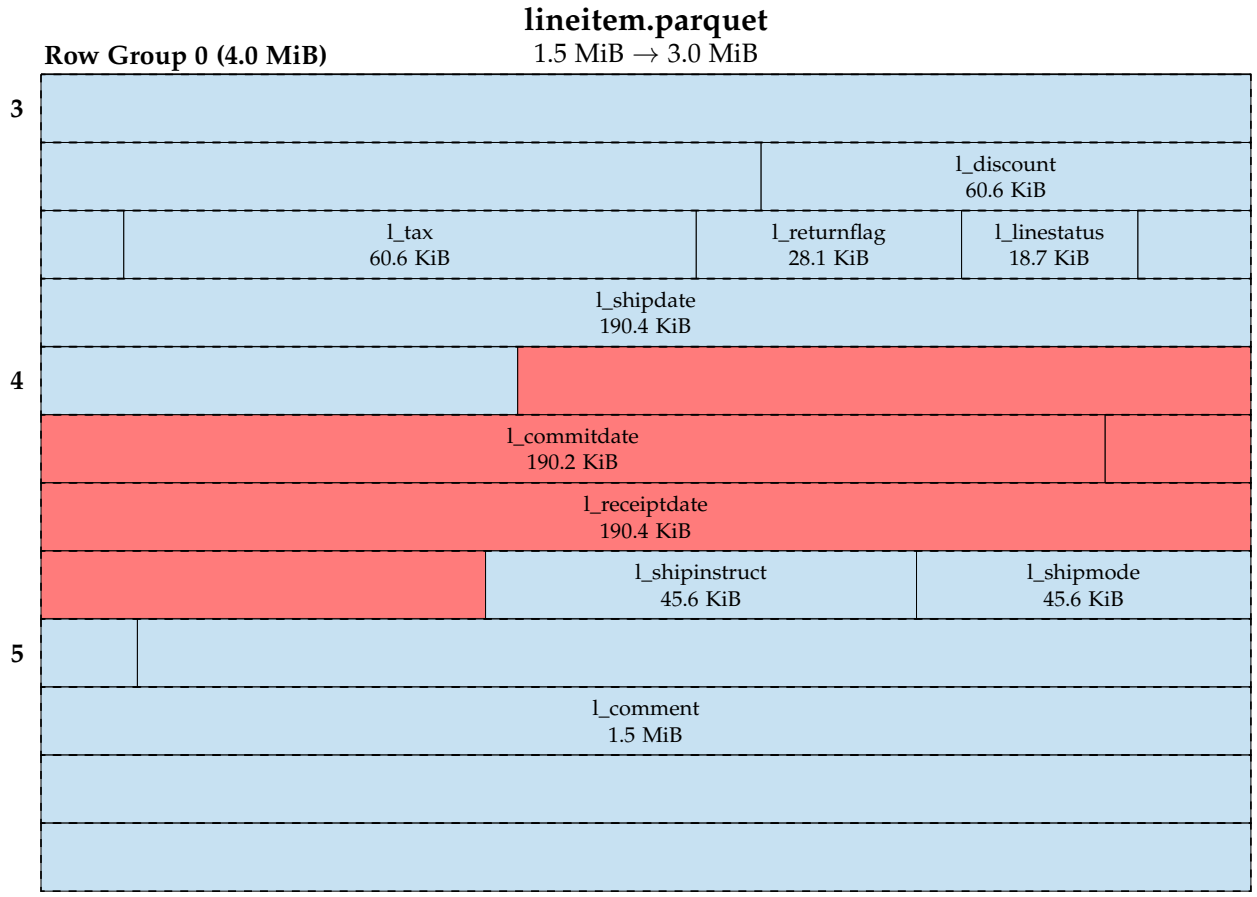


Figure 6.6: Physical layout of `lineitem.parquet` for the byte range from 1.5 MiB to 3.0 MiB, showing row group 0. Column chunks accessed by TPC-H Q21 (l_{suppkey}, l_{orderkey}, $l_{\text{receiptdate}}$, and $l_{\text{commitdate}}$) are shown in red, while all other column chunks are shown in light blue. Each line represents 128 KiB and every four lines form a 0.5 MiB address-space block.

6.4.2 Improved Cache Memory Utilization

As shown in the previous section, column chunk-based caching can reduce the amount of data that has to be read from persistent storage. However, the resulting performance improvement is often limited because the reduction in transferred data can be counteracted by the additional read operations required to access the individual column chunks. The smaller amount of data transferred therefore does not necessarily translate into a similarly large reduction in execution time.

However, the reduced amount of data has a more significant effect on the utilization of the cache memory. Since column chunk-based caching stores only the column chunks that are required by a query, less cache memory is needed to retain the same set of useful data. Consequently, a larger portion of the query's working set can fit into a cache of a given size. This allows column chunk-based caching to achieve more cache hits without increasing the physical cache capacity.

This effect is illustrated in Figure 6.7, which compares the warm execution times for fixed-size and column chunk-based caching. In contrast to the cold execution times, the differences between the two approaches are substantially larger for several queries. For example, Q01 is reduced from 6.4 s to 1.6 s, Q03 from 11.1 s to 1.8 s, Q04 from 7.4 s to 1.5 s, and Q06 from 7.0 s to 1.2 s. Similarly, Q12 decreases from 10.3 s to 1.6 s, while Q17 and Q18 decrease from 11.9 s to 2.2 s and from 9.3 s to 3.0 s, respectively. The largest improvement can be observed for Q21, where the execution time decreases from 26.7 s to 6.4 s.

For these measurements, the cache size is limited to 20% of the dataset. Despite this restriction, the reduced storage requirements of column chunk-based caching allow the complete or substantially larger portions of the working sets of several queries to fit into the cache. Once the relevant working set is retained in the cache, subsequent executions can avoid the expensive accesses to the simulated remote storage and therefore achieve substantially lower execution times.

The results demonstrate that the main advantage of dynamic block sizes is not necessarily the reduction in the execution time of individual cold reads. Instead, their ability to store only the required portions of the data makes more efficient use of the available cache memory. This becomes particularly important when the cache is smaller than the complete dataset, as the more precise cache granularity can determine whether a query's working set fits into the available cache. Thus, column chunk-based caching can provide significant performance improvements through increased effective cache capacity even when its direct benefit for individual storage accesses is comparatively small.

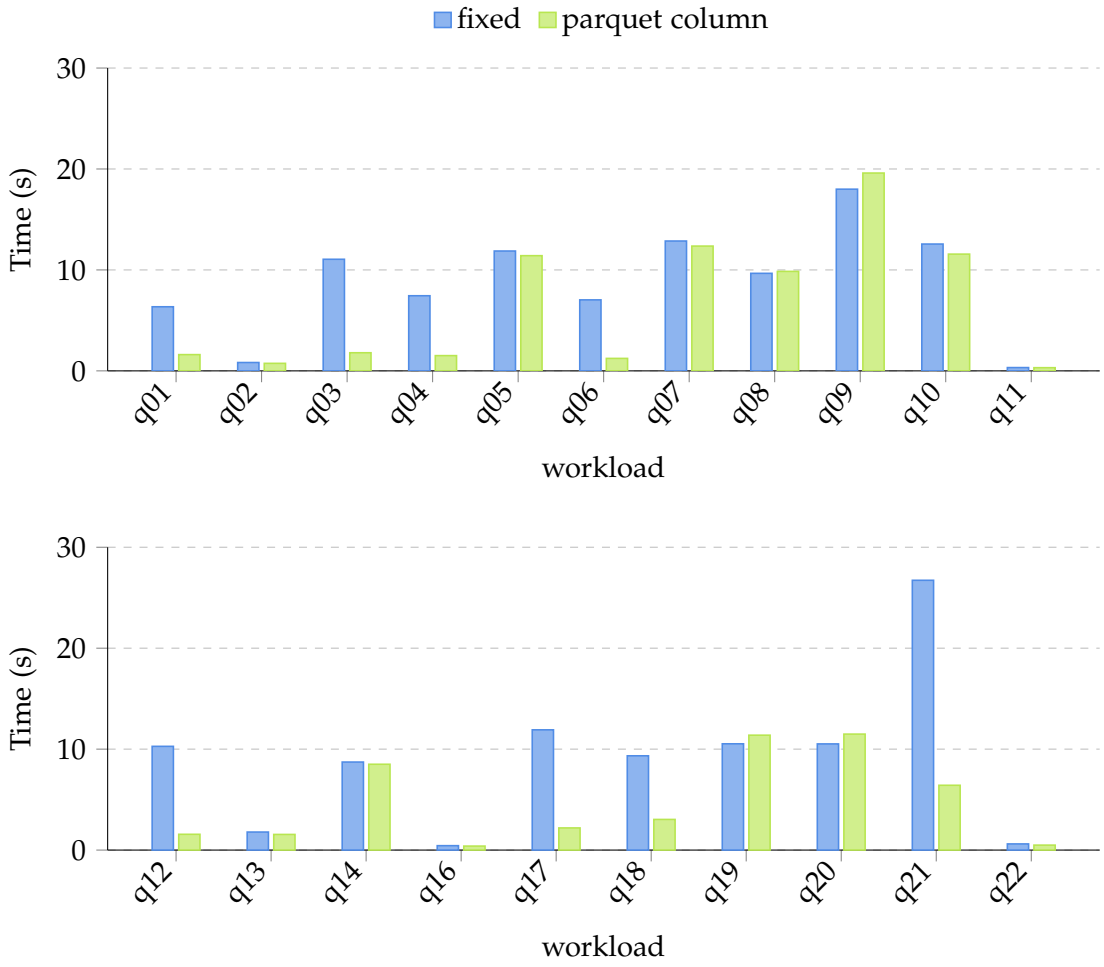


Figure 6.7: Warm execution time for parquet column and fixed-size caching with 64 threads, a block size of 512 KiB for fixed blocks, LRU replacement, a cache size of 20% of the dataset, and the TPC-H SF100 benchmark. Simulated storage has a bandwidth of 0.5 Gbit/s and a latency of 20 ms.

6.4.3 Evaluation on Query Sequences

The performance improvements observed for individual queries also carry over to sequences of queries, which provide a more realistic representation of database workloads. In such workloads, the cache contents are shared between consecutive queries, making the ability to retain relevant data in the limited cache space particularly important. Figures 6.8 and 6.9 show the warm execution times for sequences of three and five TPC-H queries, respectively.

For sequences of three queries, the results in Figure 6.8 show a similar behavior to that observed for individual queries. Column chunk-based caching provides substantial improvements for several workloads. For example, the execution time of `q04q16q22` is reduced from 10.2 s to 2.3 s, while `q06q01q06` decreases from 20.4 s to 4.2 s. The sequence `q06q06q01` shows a similar reduction, from 20.5 s to 4.1 s. In contrast, when fixed-size blocks perform better, the difference is generally comparatively small. For example, `q01q09q02` takes 26.1 s with fixed-size blocks compared to 28.3 s with column chunk-based caching, and `q07q09q20` differs by only approximately 1.5 s. This indicates that the additional read overhead (in terms of numbers of reads, not bytes) of column chunk-based caching can cause a moderate performance penalty in some workloads, while its improved cache utilization can result in substantially larger gains in workloads where it is beneficial.

The advantage of column chunk-based caching becomes more pronounced as the query sequences become longer. Figure 6.9 shows the results for sequences of five queries. Column chunk-based caching outperforms fixed-size blocks for the large majority of the evaluated sequences. For example, `q04q08q12q17q21` decreases from 66.4 s to 43.6 s, while `q02q05q14q18q21` decreases from 63.8 s to 42.0 s. A particularly large improvement can be observed for the repeated-query sequence `q06q01q06q01q06`, where the execution time is reduced from 33.8 s to 6.9 s.

The increasing advantage for longer sequences can be explained by the improved utilization of the available cache memory. As more queries are executed, the cache is exposed to a larger and more diverse working set. With fixed-size blocks, data that is not required by the workload can occupy cache space due to the coarse granularity of the cache entries. Column chunk-based caching can use the same cache capacity more efficiently by retaining only the required columns of the Parquet files. The resulting additional effective cache capacity becomes increasingly valuable as the number of queries and the size of the combined working set increase.

Overall, the results show that the benefits of column chunk-based caching are not limited to isolated queries. They persist for more realistic sequences of queries and become more apparent as the sequences grow longer. While fixed-size blocks can still provide slightly better performance for individual workloads due to their lower number

of read operations, these disadvantages are generally smaller than the improvements obtained from the more efficient use of cache memory when column chunk-based caching is beneficial.

6.5 Size-aware Replacement Policies

The previous sections have shown that column chunk-based caching can improve query execution time compared to fixed-size block caching, primarily because the more precise cache granularity allows the available cache memory to be used more efficiently. A further question is whether incorporating the size of individual cache entries into replacement decisions can improve cache performance. To investigate this, the conventional policies will be compared with their size-aware variants.

6.5.1 LRU vs. Size-Aware LRU

The size-aware LRU policy modifies the eviction behavior of LRU by taking the size of each cached block into account. Large blocks are evicted more readily, while smaller blocks are allowed to remain in the cache for additional rounds before they become eviction candidates. This allows a larger number of small blocks to be retained within the same cache capacity.

Figure 6.10 shows the resulting warm execution times for the evaluated sequences of three queries. The size-aware policy improves the execution time for most workloads. Particularly large improvements can be observed for `q03q05q07`, where the execution time decreases from 31.1 s to 20.4 s, and for `q19q14q19`, where it decreases from 31.3 s to 23.3 s. Similarly, `q02q16q20` decreases from 13.5 s to 8.1 s and `q07q09q20` from 42.3 s to 37.5 s. For some workloads, the two policies are indistinguishable at the precision reported here, such as `q04q16q22`, where both configurations require approximately 2.3 s.

The increased performance can be explained by the higher number of cache hits shown in Figure 6.11. For most workloads, the size-aware policy results in substantially more cache hits than conventional LRU. For example, `q01q09q02` increases from 8,825 to 55,625 cache hits, while `q03q05q07` increases from 13,604 to 84,434 hits. An even larger difference can be observed for `q19q14q19`, where the number of cache hits increases from 13,383 to 90,706. Similar improvements are visible for `q07q09q20`, `q08q12q19`, and `q10q13q21`.

The higher number of cache hits results from the different treatment of cache entries based on their size. With conventional LRU, a large block can occupy a substantial portion of the cache even though it represents only a single cache entry. Evicting such a block only when it reaches the end of the LRU order can therefore prevent several

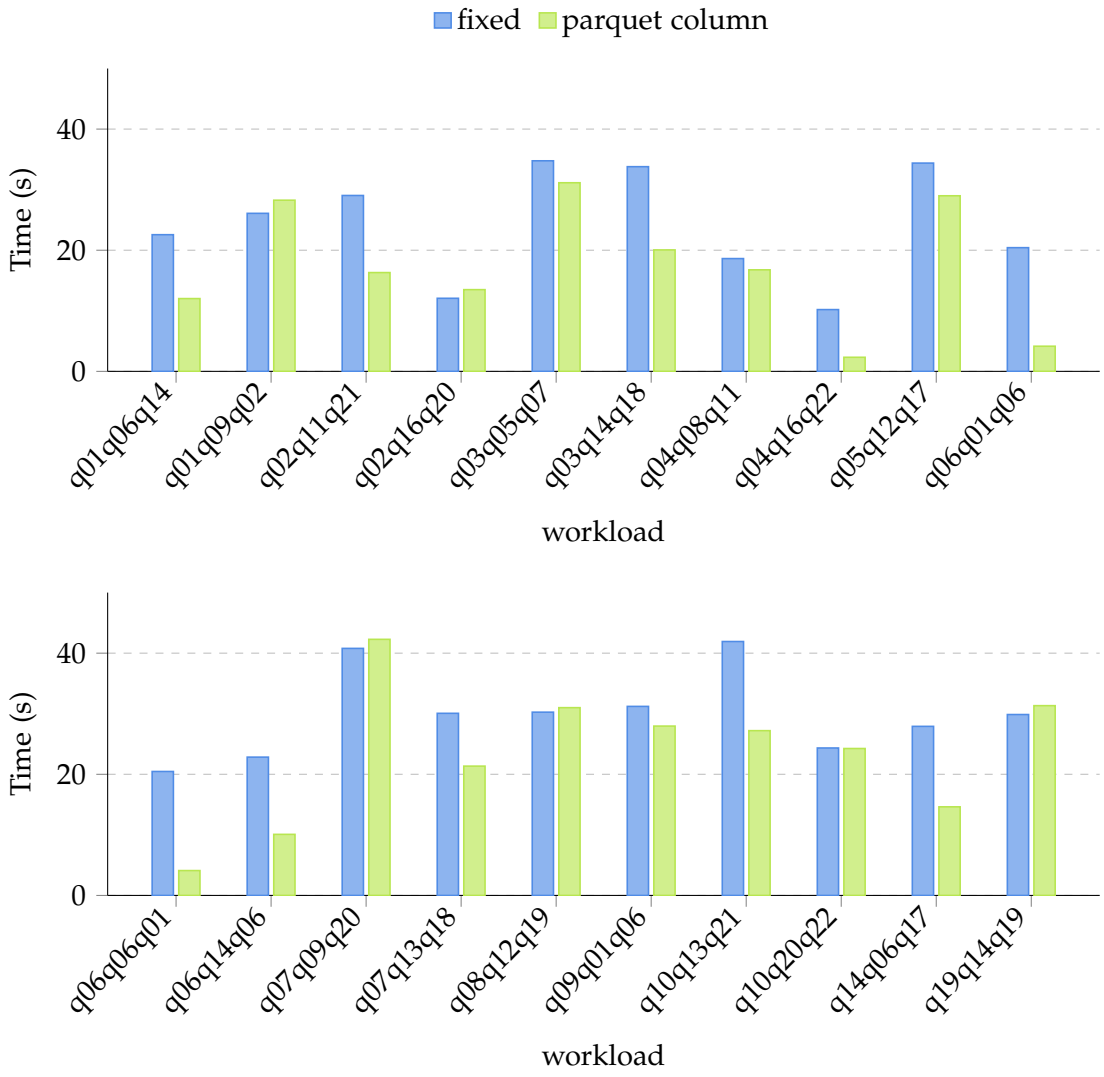


Figure 6.8: Warm execution time for fixed-size and column-chunk-based caching for sequences of three TPC-H queries, using 64 threads, a fixed block size of 512 KiB, LRU replacement, a cache size of 20% of the dataset, and the TPC-H SF100 benchmark. Simulated storage has a bandwidth of 0.5 Gbit/s and a latency of 20 ms.

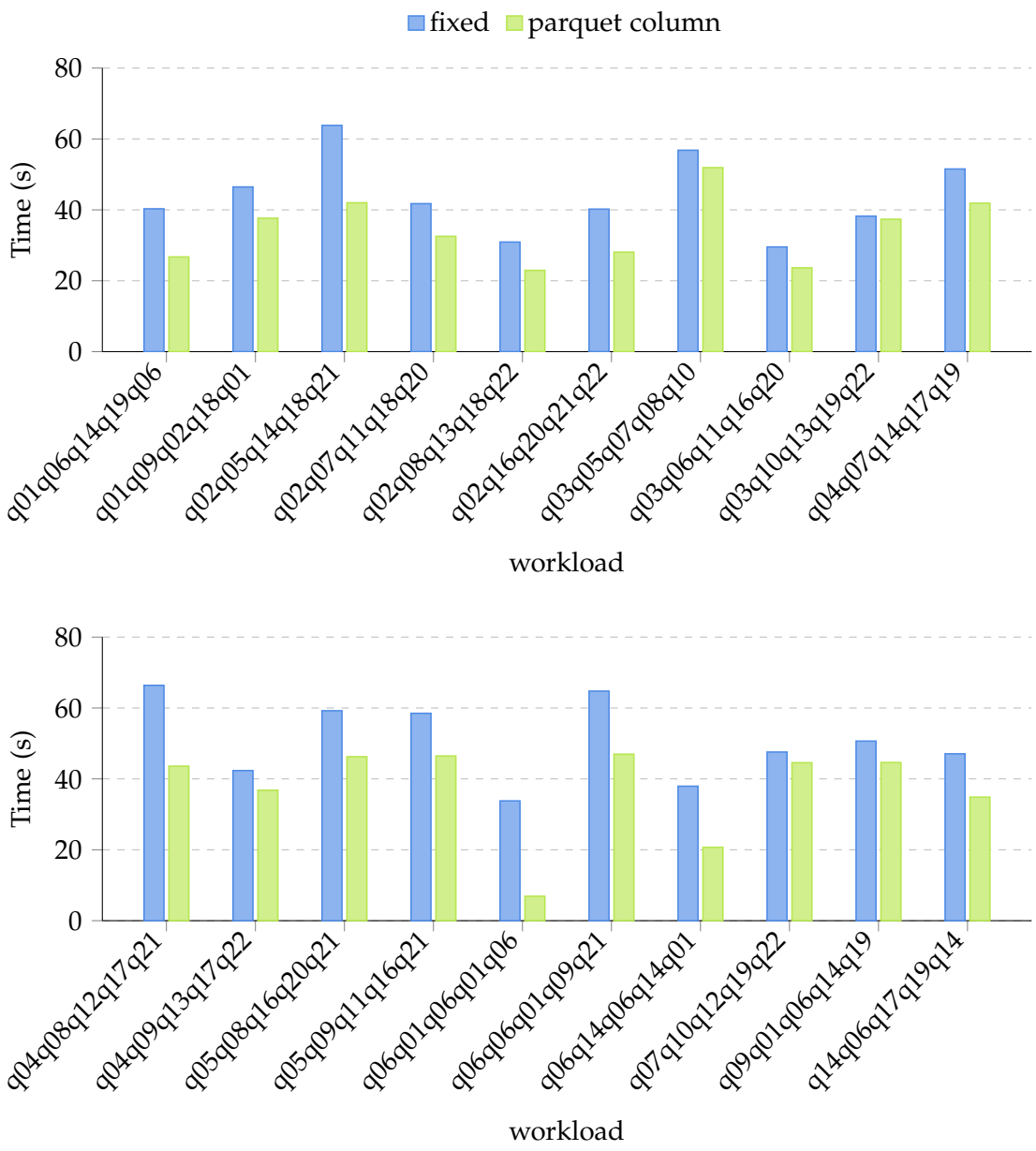


Figure 6.9: Warm execution time for fixed-size and column-chunk-based caching for sequences of five TPC-H queries, using 64 threads, a fixed block size of 512 KiB, LRU replacement, a cache size of 20% of the dataset, and the TPC-H SF100 benchmark. Simulated storage has a bandwidth of 0.5 Gbit/s and a latency of 20 ms.

smaller blocks from being retained simultaneously. The size-aware policy instead favors retaining smaller blocks by allowing them to survive additional eviction rounds, while large blocks reach eviction more quickly. As a result, more individual blocks can be held in the available cache memory, increasing the probability that subsequent accesses can be served directly from the cache.

The results therefore indicate that cache entry size is useful information for replacement decisions when variable-sized column chunks are used. Taking the block size into account allows the limited cache capacity to accommodate more cache entries and consequently increases the number of cache hits. The higher number of cache hits in turn reduces accesses to persistent storage and leads to lower query execution times for many of the evaluated workloads.

6.5.2 LRU vs. S3-FIFO

The previous evaluation showed that taking the size of cache entries into account can improve the performance of LRU. However, LRU is a relatively simple replacement policy and has known limitations for certain access patterns. In particular, LRU is not scan-resistant: a sequential scan can replace entries that are still useful with data that is accessed only once during the scan. Therefore, a more sophisticated replacement policy was also evaluated. For this purpose, S3-FIFO was selected as an alternative to LRU.

Figure 6.12 compares the warm execution times of LRU and S3-FIFO for the evaluated query sequences. S3-FIFO achieves a lower execution time for most of the workloads. The largest improvements can be observed for `q03q10q13q19q22`, where the execution time decreases from 37.4 s with LRU to 21.4 s with S3-FIFO, and for `q03q05q07q08q10`, where it decreases from 51.9 s to 37.4 s. Further substantial improvements can be observed for `q05q09q11q16q21`, with a reduction from 46.5 s to 32.3 s, and `q07q10q12q19q22`, with a reduction from 44.6 s to 32.6 s. In contrast, there are also workloads for which S3-FIFO provides no benefit. For `q06q01q06q01q06`, the two policies are indistinguishable at the precision reported here, both requiring approximately 6.9 s, while `q06q06q01q09q21` shows a modest increase from 47.0 s to 49.1 s.

The results therefore confirm that a more sophisticated replacement policy can significantly improve performance. S3-FIFO is able to outperform LRU for a wide range of workloads, demonstrating the benefit of using a replacement policy with more sophisticated admission and eviction behavior.

6.5.3 S3-FIFO vs. Size-Aware S3-FIFO

Since the previous evaluation showed that considering the size of cache entries can further improve LRU, the same idea was applied to S3-FIFO. Three approaches were

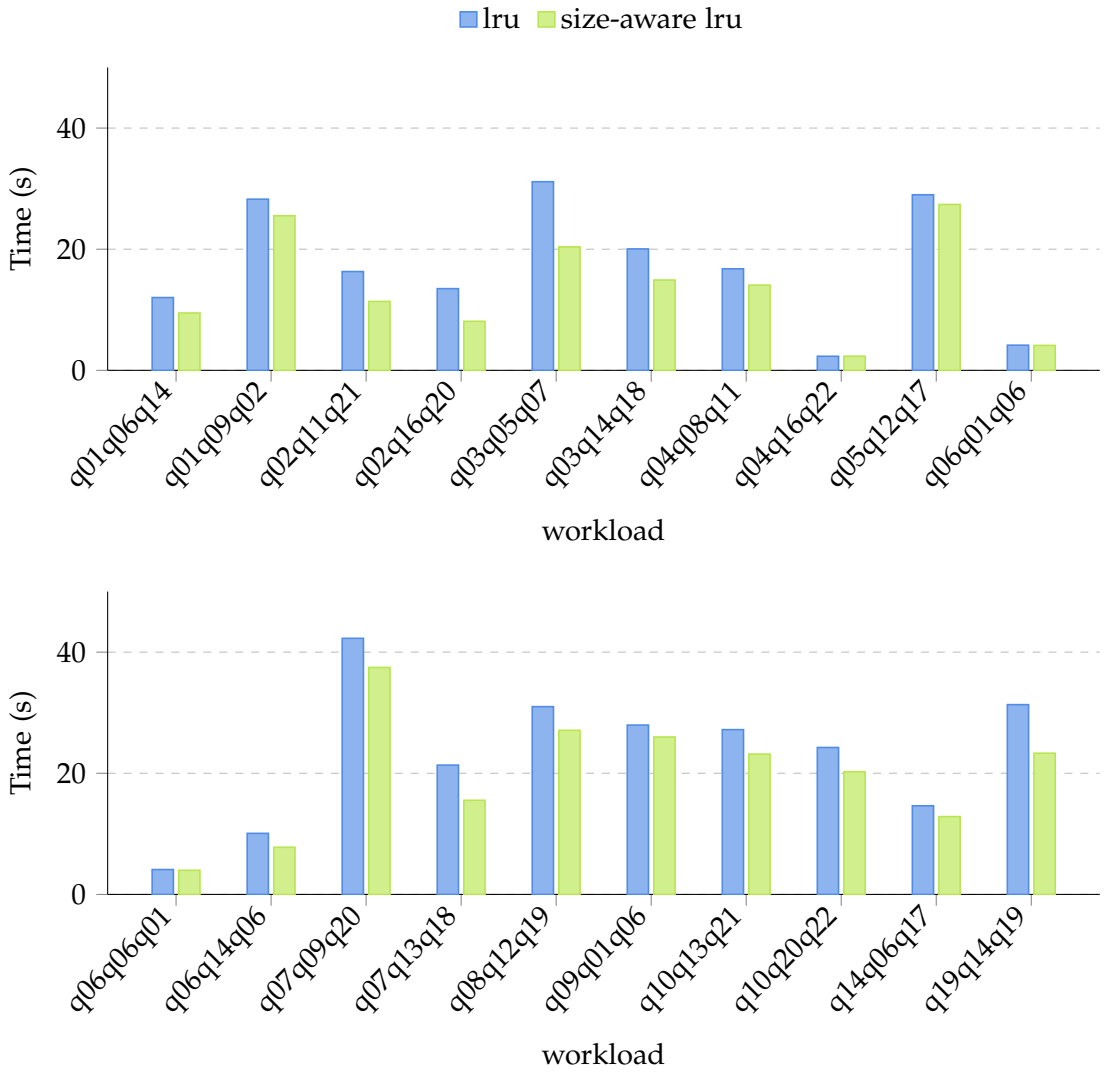


Figure 6.10: Warm execution time for LRU and Size-Aware LRU replacement policies with 64 threads, parquet column caching, a cache size of 20% of the dataset, and the TPC-H SF100 benchmark. Simulated storage has a bandwidth of 0.5 Gbit/s and a latency of 20 ms.

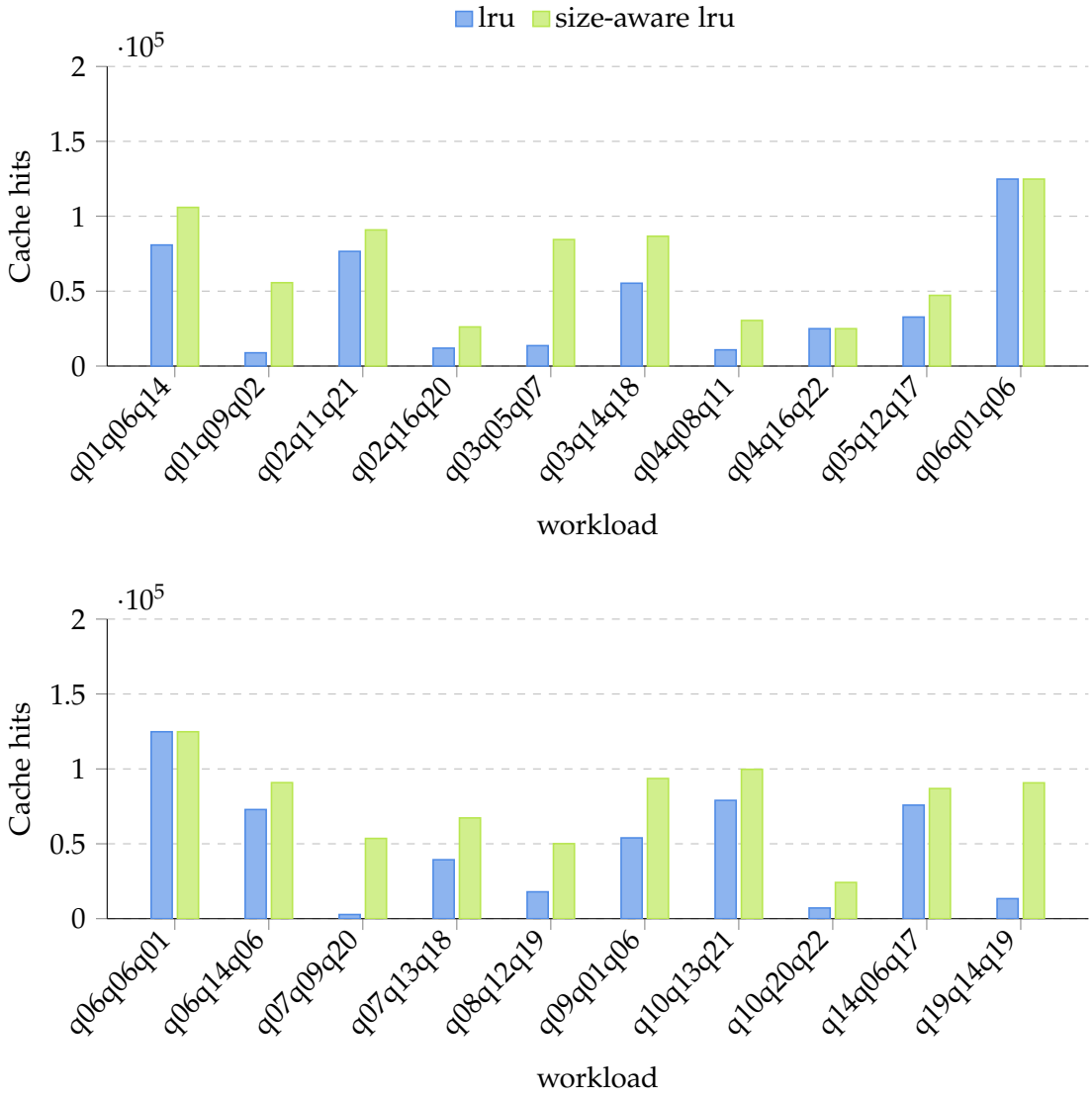


Figure 6.11: Cache hits for LRU and Size-Aware LRU replacement policies with 64 threads, parquet column caching, a cache size of 20% of the dataset, and the TPC-H SF100 benchmark. Simulated storage has a bandwidth of 0.5 Gbit/s and a latency of 20 ms.

evaluated. The first, referred to as S3-FIFO promotion, takes the block size into account when promoting an entry from the small FIFO queue to the main FIFO queue. The second, referred to as S3-FIFO rounds, gives smaller blocks additional rounds in the main FIFO queue before they become eviction candidates. The third combines both mechanisms. Figure 6.13 compares these three variants.

The results show that giving smaller blocks additional rounds in the main FIFO queue provides the best overall performance. The S3-FIFO rounds variant achieves lower execution times than the promotion variant for most workloads. For example, for `q05q08q16q20q21`, the execution time is reduced from 38.5 s with promotion to 31.4 s with rounds. For `q06q06q01q09q21`, the reduction is even larger, from 49.4 s to 38.0 s. Similarly, `q14q06q17q19q14` decreases from 29.0 s to 23.0 s. Combining the two mechanisms does not provide an additional benefit. In many cases, the combined variant performs slightly worse than using rounds alone.

Finally, Figure 6.14 directly compares standard S3-FIFO with the size-aware variant using additional rounds. The size-aware variant improves the execution time for the majority of the workloads. Particularly large improvements can be observed for `q06q06q01q09q21`, where the execution time decreases from 49.1 s to 38.0 s, and for `q05q08q16q20q21`, where it decreases from 38.4 s to 31.4 s. The sequence `q14q06q17q19q14` also benefits substantially, decreasing from 29.0 s to 23.0 s. While a few workloads show a small regression, such as `q06q14q06q14q01`, the overall results indicate that accounting for block size through additional rounds can further improve S3-FIFO.

These results demonstrate that the benefits of size-aware replacement are not specific to LRU. S3-FIFO can also benefit from considering the size of cached blocks, with the additional-rounds mechanism providing the most effective approach among the evaluated variants. This suggests that cache entry size is a useful signal for replacement decisions across different classes of replacement policies.

6.6 Overall Comparison

The previous evaluations have considered the individual contributions of the proposed techniques. First, dynamic block sizes were shown to improve the utilization of the available cache memory compared to fixed-size blocks. Second, size-aware LRU increased the number of cache hits for most evaluated workloads, while size-aware variants of both LRU and S3-FIFO reduced execution times for many evaluated workloads. Finally, S3-FIFO was shown to outperform LRU for a wide range of workloads, and giving smaller blocks additional rounds in the main FIFO queue further improved its performance.

Based on these results, the best-performing combination of the evaluated techniques

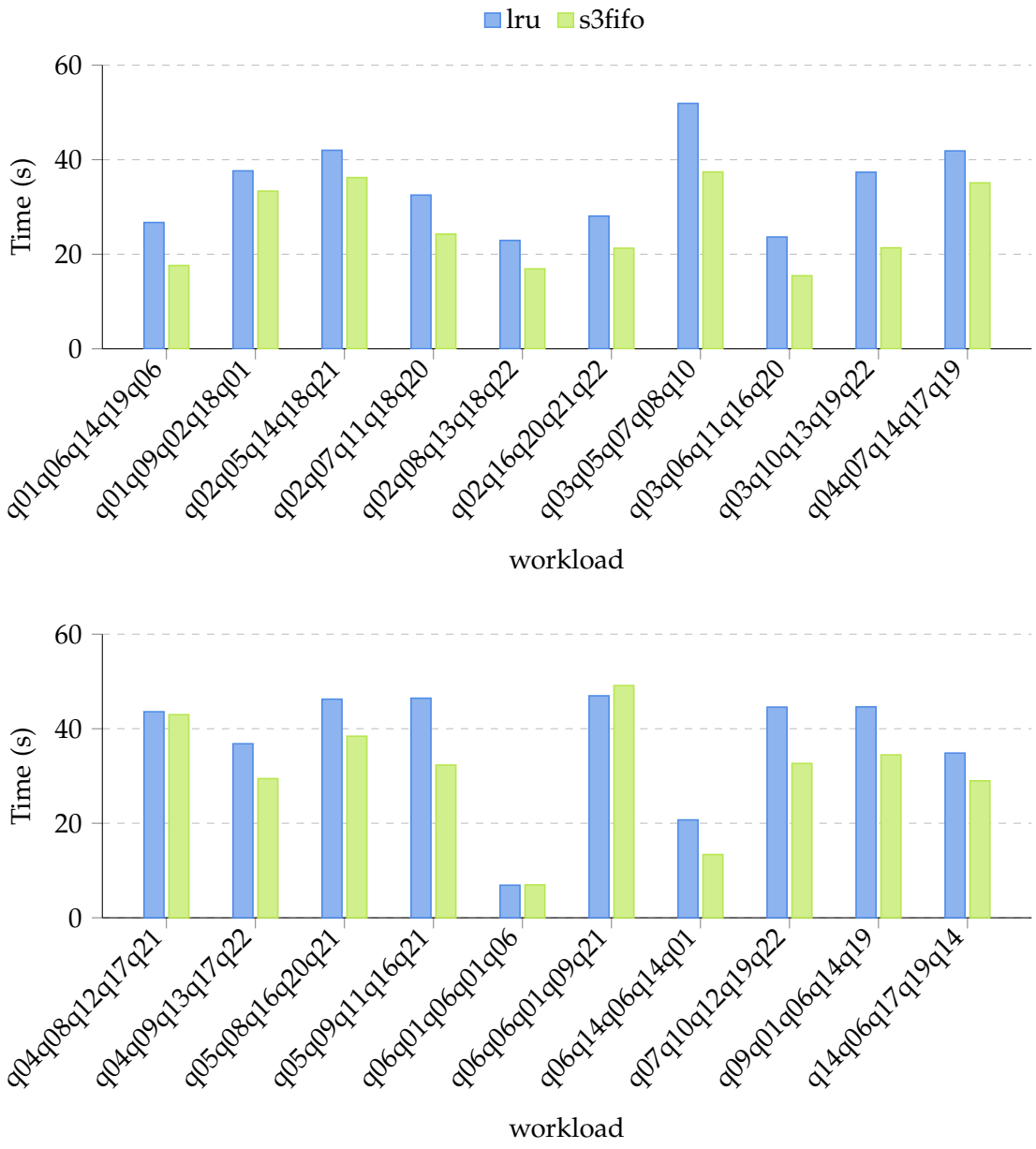


Figure 6.12: Warm execution time for LRU and S3-FIFO replacement policies with 64 threads, parquet column caching, a cache size of 20% of the dataset, and the TPC-H SF100 benchmark. Simulated storage has a bandwidth of 0.5 Gbit/s and a latency of 20 ms.

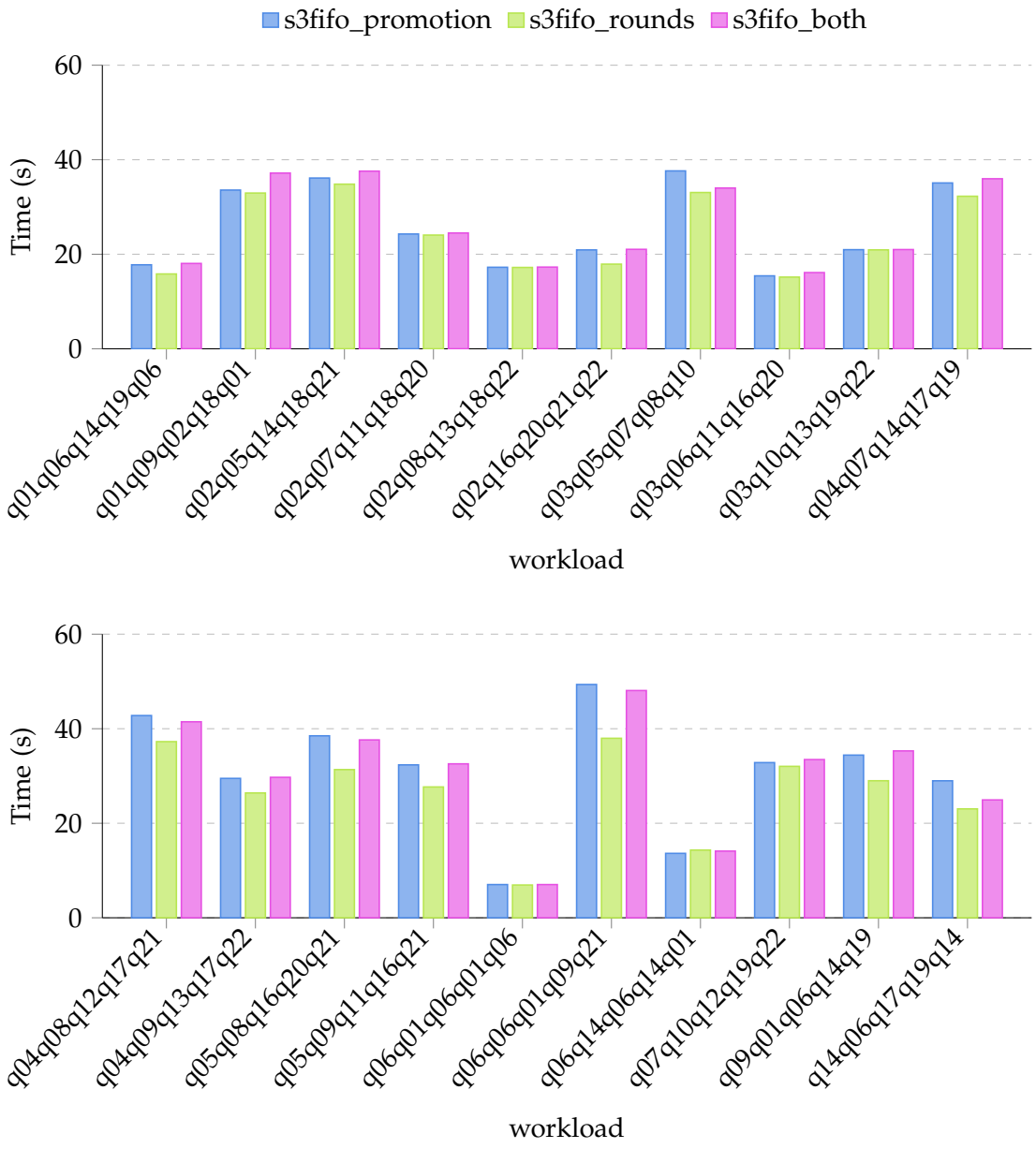


Figure 6.13: Warm execution time for S3-FIFO promotion, S3-FIFO rounds, and S3-FIFO both with 64 threads, parquet column caching, a cache size of 20% of the dataset, and the TPC-H SF100 benchmark. Simulated storage has a bandwidth of 0.5 Gbit/s and a latency of 20 ms.

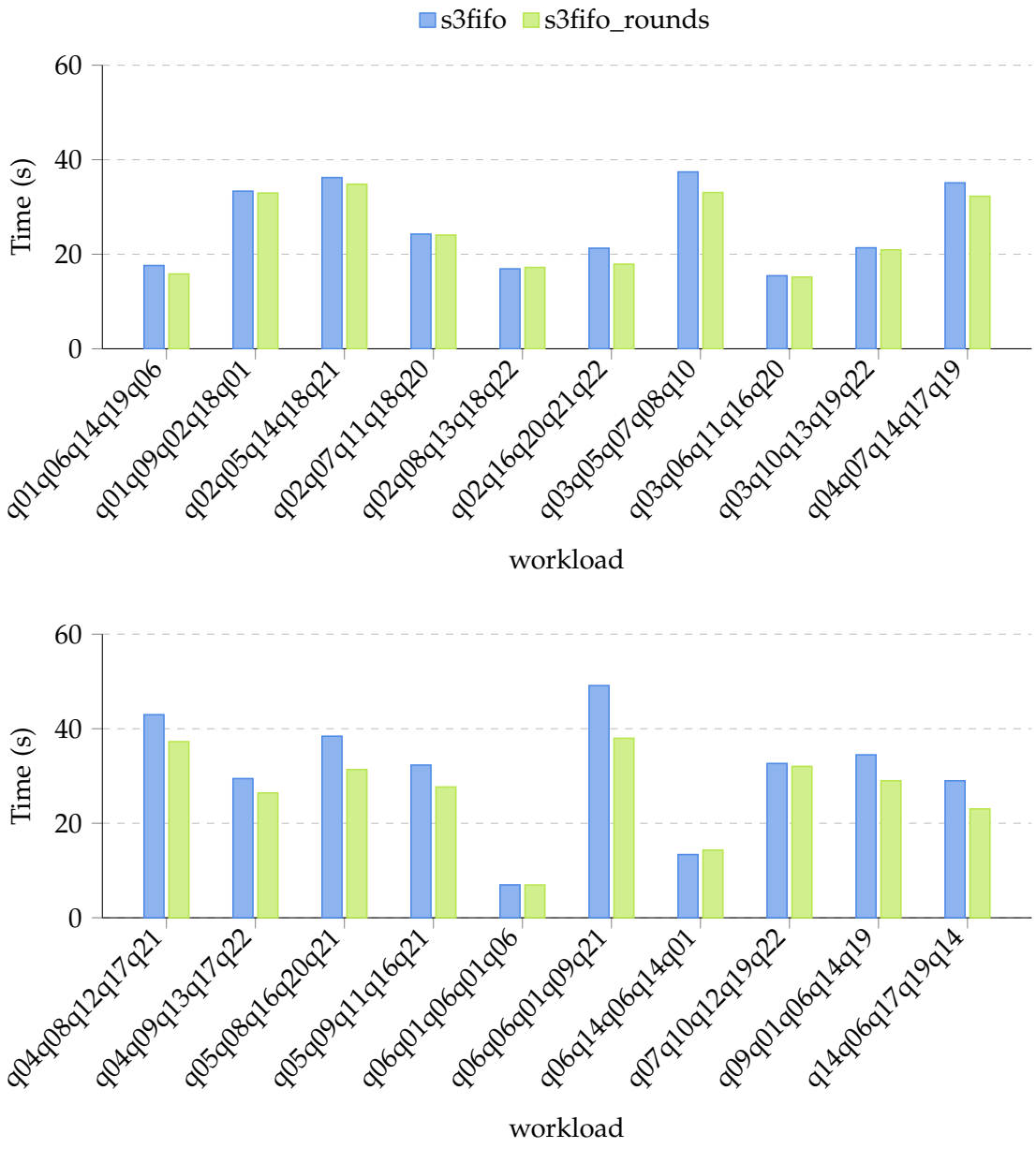


Figure 6.14: Warm execution time for S3-FIFO and S3-FIFO Rounds with 64 threads, parquet column caching, a cache size of 20% of the dataset, and the TPC-H SF100 benchmark. Simulated storage has a bandwidth of 0.5 Gbit/s and a latency of 20 ms.

is compared against the baseline configuration. The proposed configuration uses dynamic block sizes based on Parquet column chunks together with the size-aware S3-FIFO policy using additional rounds. The baseline uses fixed-size cache blocks together with conventional LRU replacement. Figure 6.15 shows the warm execution times of both configurations for the evaluated sequences of five queries.

The results demonstrate a substantial reduction in execution time when combining the proposed techniques. The total execution time across all evaluated workloads decreases from 947.8 s for the baseline to 510.2 s for the proposed configuration. This corresponds to a reduction of approximately 46% in total execution time. The average execution time per workload decreases from 47.4 s to 25.5 s.

The improvement is observed across all evaluated workloads. The smallest improvement is achieved for `q01q09q02q18q01`, where the execution time decreases from 46.5 s to 33.0 s, corresponding to a factor of approximately 1.41. The largest improvement occurs for `q06q01q06q01q06`, where execution time decreases from 33.8 s to 6.9 s, corresponding to a factor of approximately 4.87. Other substantial improvements can be observed for `q02q05q14q18q21`, which decreases from 63.8 s to 34.8 s, and `q05q09q11q16q21`, which decreases from 58.5 s to 27.7 s.

Table 6.2 summarizes the aggregate results. The arithmetic mean of the per-workload speedups is 2.03, while the ratio of total execution times is approximately 1.86, corresponding to a 46.2% reduction in total execution time. The fact that the proposed configuration outperforms the baseline for every evaluated workload indicates that the individual improvements observed in the preceding experiments complement each other when combined.

Table 6.2: Comparison of the baseline and proposed cache configurations across the 20 evaluated workloads. Per-workload speedup is the baseline execution time divided by the S3-FIFO rounds execution time. Aggregate speedup is the ratio of total execution times.

Metric	**Value**
Total time, LRU with fixed blocks	947.8 s
Total time, S3-FIFO rounds with dynamic blocks	510.2 s
Average time, LRU with fixed blocks	47.4 s
Average time, S3-FIFO rounds with dynamic blocks	25.5 s
Maximum per-workload speedup	4.87×
Minimum per-workload speedup	1.41×
Arithmetic mean of per-workload speedups	2.03×
Aggregate speedup (ratio of total times)	1.86×
Reduction in total execution time	46.2%

The overall results therefore show that the proposed approach can provide significant performance improvements over the conventional fixed-block LRU baseline. While each individual technique addresses a different limitation of the baseline, their combination results in a cache that uses its available memory more effectively, retains useful data more successfully, and consequently reduces the amount of expensive storage access required during query execution.

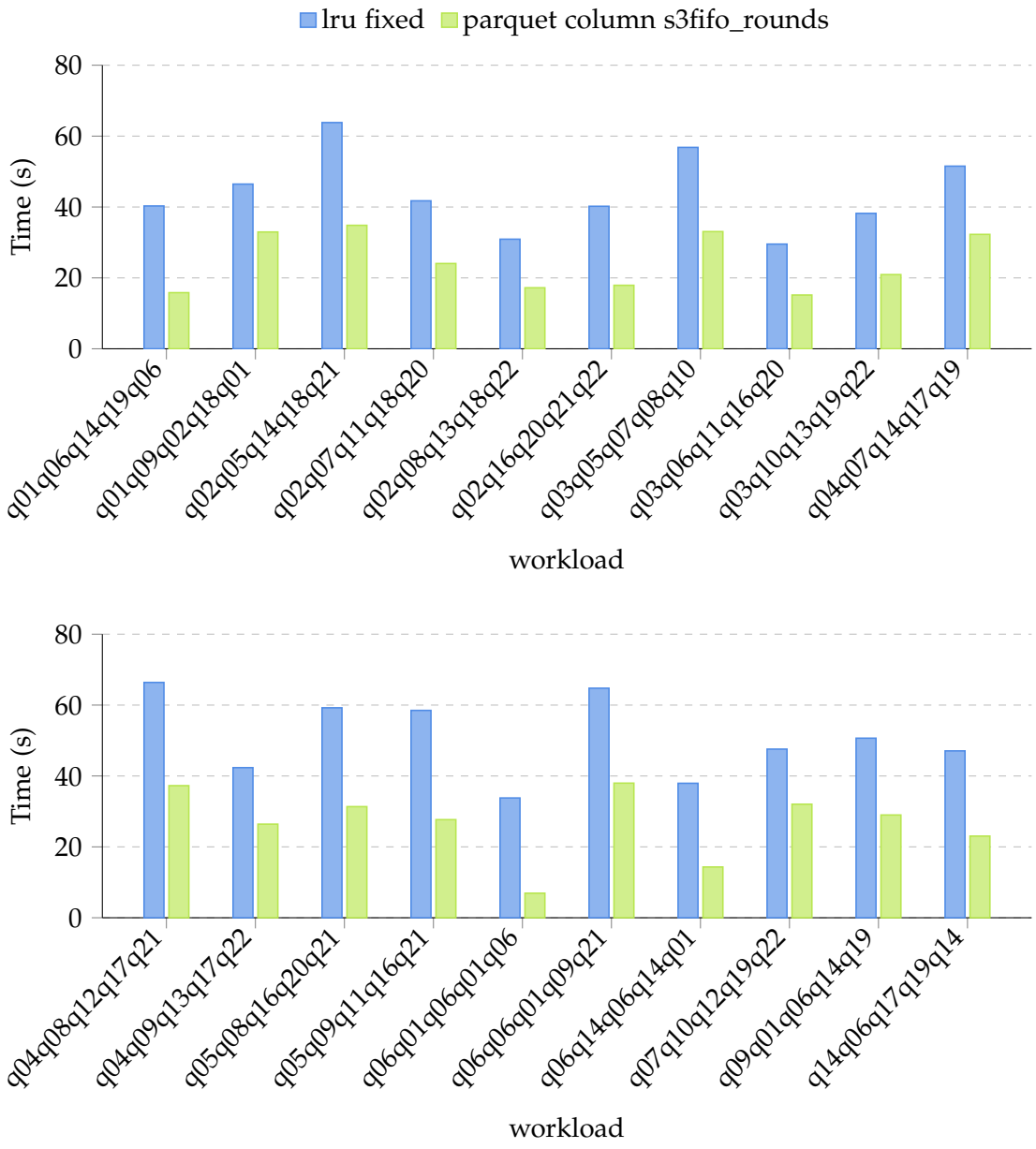


Figure 6.15: Warm execution time for fixed-size lru and parquet column s3fifo_rounds caching with 64 threads, a block size of 512 KiB for fixed blocks, a cache size of 20% of the dataset, and the TPC-H SF100 benchmark. Simulated storage has a bandwidth of 0.5 Gbit/s and a latency of 20 ms.

7 Related work

7.1 Caching with Variable-Sized Blocks

The use of variable-sized objects in caching systems has been investigated in several contexts. One example is web proxy caching, where cached objects naturally have different sizes. Cao and Irani introduced GreedyDual-Size, a replacement algorithm specifically designed for this setting [CI97]. In contrast to conventional replacement policies that treat all cached objects equally, GreedyDual-Size incorporates both the size of an object and the cost of retrieving it into its replacement decisions. The algorithm therefore considers not only whether an object is likely to be accessed again, but also the amount of cache space it occupies and the cost that can be avoided by retaining it. The authors evaluate the approach using web proxy traces and show that incorporating size and retrieval cost can improve several caching metrics [CI97]. This work demonstrates that considering the size and retrieval cost of cache entries can be beneficial when the cached objects are not uniform in size.

Variable-sized caching is also used in the OpenZFS Adaptive Replacement Cache (ARC) implementation. While the original ARC algorithm was designed for caches containing uniformly sized pages [MM03], the OpenZFS implementation operates on buffers of varying sizes and accounts for their size when managing the cache [Ope]. The implementation therefore provides an example of how a replacement policy originally designed around fixed-size cache entries can be adapted to a system in which cached objects have different sizes.

The approach investigated in this thesis differs from these systems in the type of data being cached and in how the variable sizes arise. Instead of arbitrary web objects or filesystem buffers, the cache entries correspond to Parquet column chunks. Their sizes are determined by the physical layout and encoding of the Parquet file. This makes the size of a cache entry directly dependent on the structure of the analytical data.

7.2 Replacement Policies Beyond LRU

LRU is widely used as a baseline cache replacement policy due to its simplicity and generally good performance. However, its reliance primarily on recency can make

it less effective for workloads with access patterns that are not well represented by recency alone. Therefore, this thesis also investigates the more sophisticated S3-FIFO replacement policy, which incorporates additional information about access frequency. Other replacement policies could likewise be investigated to determine whether they provide further performance improvements.

Megiddo and Modha introduced ARC, which dynamically balances recency and frequency when making replacement decisions [MM03]. ARC maintains separate structures for recently accessed and frequently accessed entries and continuously adapts the relative importance assigned to the two types of access history. This allows the policy to adapt to changing workloads without requiring a workload-specific parameter to be selected in advance. The authors report improved performance over LRU for a range of cache sizes and workloads [MM03]. ARC is particularly relevant to this thesis because it demonstrates how a replacement policy can adapt its behavior to the observed workload rather than relying on a fixed replacement strategy.

Another approach is Low Inter-reference Recency Set (LIRS), proposed by Jiang and Zhang [JZ02]. LIRS uses inter-reference recency to distinguish between blocks with different levels of locality. Instead of considering only which block was accessed most recently, it uses information about the number of distinct accesses occurring between consecutive references to an item. This allows LIRS to identify blocks with poor locality and avoid retaining them at the expense of blocks that exhibit stronger reuse. The authors specifically motivate LIRS by limitations of LRU for workloads with weak locality and show improvements in buffer cache performance for such workloads [JZ02].

These approaches demonstrate that replacement decisions can benefit from information beyond simple recency. The replacement policies investigated in this thesis follow the same general motivation but additionally consider the variable size of Parquet column chunks. In particular, the size-aware policies combine information about access behavior with the amount of cache capacity occupied by an entry. This is relevant because two blocks with identical access histories can have very different effects on cache utilization when their sizes differ.

8 Summary

This thesis investigated file-layout-aware caching and size-aware cache replacement for cloud-native analytical query engines. Cloud-native database systems separate compute and storage, which allows these resources to be scaled independently but can make data access more expensive when data has to be retrieved from remote persistent storage. Caching can reduce this cost by retaining previously accessed data locally. However, conventional caches commonly use fixed-size blocks, which do not necessarily correspond to the structure of the underlying data.

To address this limitation, a column chunk-based caching approach was developed for Parquet files. Instead of dividing files into fixed-size blocks, the cache can use Parquet column chunks as cache entries. This allows the cache granularity to follow the layout of the underlying file and can reduce the amount of data that has to be transferred and stored when an analytical query accesses only selected columns. At the same time, column chunks have variable sizes, which creates additional challenges for cache replacement. Therefore, multiple replacement policies were implemented and extended to take the size of cache entries into account.

The proposed approach was implemented in DuckDB using an existing remote filesystem caching extension as its foundation. The implementation supports both fixed-size blocks and column chunk-based caching, allowing the two approaches to be evaluated under the same conditions. An interchangeable replacement policy interface was introduced, through which conventional policies such as LRU and S3-FIFO as well as their size-aware variants could be evaluated. The size-aware policies investigated different ways of incorporating block size into replacement decisions, including giving smaller blocks additional replacement opportunities.

The evaluation was conducted using TPC-H workloads with simulated remote storage latency and bandwidth. The results show that column chunk-based caching can improve cache utilization and reduce query execution time compared with fixed-size caching. However, the benefit depends on the access pattern of the workload. Fixed-size blocks can sometimes provide useful prefetching by including nearby data in a single cache block, while column chunk-based caching can avoid transferring data that is not required by the query. By making more efficient use of the available cache memory, column chunk-based caching increasingly outperforms fixed-size caching as the length of the query sequence increases.

The evaluation of replacement policies further shows that accounting for the size of cache entries can improve cache performance when variable-sized blocks are used. In particular, the size-aware S3-FIFO variant that provides smaller blocks with additional replacement rounds consistently improved upon the standard S3-FIFO policy for most evaluated query sequences. Combining column chunk-based caching with this size-aware replacement policy resulted in substantial improvements compared with the conventional baseline using fixed-size blocks and LRU replacement.

Overall, the results demonstrate that using the layout of the underlying Parquet files as the basis for cache granularity is a viable alternative to conventional fixed-size caching for the evaluated analytical workloads. The experiments also show that variable-sized cache entries benefit from replacement policies that explicitly account for their sizes. Together, file-layout-aware caching and size-aware replacement provide a more efficient way of using limited cache capacity for cloud-native analytical query processing.

9 Future work

This thesis has shown that caching based on Parquet column chunks can provide significant performance improvements compared to conventional fixed-size cache blocks. In particular, column chunk-based caching allows the cache to store less data that is not required by a query, which can improve the effective utilization of the available cache memory. The evaluation also showed that the benefits depend on the workload and the characteristics of the storage system. Further investigation of column chunk-based caching could therefore provide several opportunities for improving both its performance and adaptability.

One possible direction is to investigate a zero-copy implementation of column chunk-based caching. The current implementation requires cached data to be copied between the cache and the database system. Eliminating these copies could further reduce the overhead of serving data from the cache and therefore provide an additional performance improvement. However, a zero-copy design would also make the lifetime management of cached blocks more complex. The cache would need to ensure that a block remains valid for as long as it is being accessed by the database system. Furthermore, such an implementation would result in a tighter coupling between the cache and the DBMS, as the cache would need to interact more directly with the memory management and data access mechanisms of the query engine. Investigating this trade-off between performance and architectural independence would therefore be valuable.

A second direction concerns the number of individual read operations required when using column chunks. As shown in Section 6.4, dynamic block sizes do not outperform fixed-size blocks for every query execution. In particular, when multiple required column chunks are located close to each other within a Parquet file, fixed-size caching can implicitly prefetch these chunks by fetching the surrounding data in a single read. Column chunk-based caching, in contrast, reads each required chunk individually. Although this avoids transferring data that is not required by the query, it can result in multiple storage requests, each incurring the per-request latency. In such cases, the additional read latency can outweigh the benefit of transferring less data. Future work could therefore investigate hybrid approaches that combine fixed-size and column chunk-based caching. For example, nearby column chunks could be combined into a single cache block when this reduces the number of read operations without

causing excessive data transfer. More generally, the cache could dynamically determine an appropriate granularity based on the physical proximity of requested chunks and the characteristics of the underlying storage system. Such an approach could retain the precise data selection of column chunk-based caching while exploiting the implicit prefetching benefits of larger reads.

The replacement policies provide another area for further investigation. This thesis evaluated LRU, S3-FIFO and size-aware variants of these policies. Other replacement algorithms could be evaluated, including more sophisticated approaches such as ARC. Such policies could also be extended to account for the variable sizes of Parquet column chunks. More generally, future work could investigate different ways of incorporating block size into replacement decisions. Instead of assigning smaller blocks a fixed number of additional rounds, a policy could consider factors such as the size of a block, its access frequency, its recency, and potentially the cost of retrieving it from persistent storage. This could allow the replacement policy to make more informed decisions about which blocks provide the greatest benefit when retained in the cache.

Finally, the parameters used by the cache policies could be made adaptive. In this thesis, parameters such as the number of additional rounds assigned to smaller blocks were selected based on the characteristics of the simulated S3 environment, particularly its latency and transfer rate. However, the optimal configuration can depend on several system parameters, including storage latency, transfer bandwidth, the number of query execution threads, and the available cache size. Instead of requiring these parameters to be selected manually, future work could investigate mechanisms that automatically adapt the cache configuration to the current environment and workload. For example, the cache could adjust the parameter that determines how the size of a cache block influences eviction decisions at runtime. This could make the cache more robust across different cloud storage environments and workloads without requiring manual configuration.

Overall, these directions could extend the approach presented in this thesis from a cache that supports workload-aware configuration towards a cache that can automatically adapt its data granularity, replacement strategy, and resource allocation to the characteristics of the workload and storage environment.

Abbreviations

DBMS Database Management System

RAM Random-Access Memory

OLAP Online Analytical Processing

LRU Least Recently Used

FIFO First In, First Out

ARC Adaptive Replacement Cache

LIRS Low Inter-reference Recency Set

NVMe Non-Volatile Memory Express

SSD Solid-State Drive

CPU Central Processing Unit

List of Figures

2.1 Overview of the physical structure of an Apache Parquet file. The file consists of row groups containing one column chunk per column. Each column chunk is divided into encoded and compressed pages. The file metadata is stored in the footer. 4
2.2 DuckDB caching architectures using cloud and local storage. 7
2.3 Approximate access latency for different storage media. 8

5.1 Enabling column chunk-based caching 15
5.2 Registering a Parquet file 15
5.3 Retrieving Parquet metadata 15
5.4 Configuring the fixed cache block size 16
5.5 General eviction policy interface 18
5.6 Distribution of column chunk sizes in TPC-H SF100 using 100 KiB bins. 21

6.1 Ratio of bytes read using column chunk-based caching to bytes read using fixed-size caching. Each group represents one TPC-H query. Lower values indicate a larger reduction in transferred data through column chunk-based caching. 30
6.2 Cold execution time for parquet column and fixed-size caching with 64 threads, a block size of 512 KiB for fixed blocks, LRU replacement, a cache size of 100% of the dataset, and the TPC-H SF100 benchmark. Simulated storage has a bandwidth of 0.5 Gbit/s and a latency of 20 ms. 32
6.3 Physical layout of `lineitem.parquet` for the byte range from 0 B to 1.5 MiB, showing row group 0. Column chunks accessed by TPC-H Q01 ($l_{\text{returnflag}}$, $l_{\text{linestatus}}$, l_{quantity}, $l_{\text{extendedprice}}$, l_{discount}, l_{tax}, and l_{shipdate}) are shown in red, while all other column chunks are shown in light blue. Each line represents 128 KiB and every four lines form a 0.5 MiB address-space block. 33

6.4 Physical layout of `lineitem.parquet` for the byte range from 1.5 MiB to 3.0 MiB, showing row group 0. Column chunks accessed by TPC-H Q01 ($l_{\text{returnflag}}$, $l_{\text{linestatus}}$, l_{quantity}, $l_{\text{extendedprice}}$, l_{discount}, l_{tax}, and l_{shipdate}) are shown in red, while all other column chunks are shown in light blue. Each line represents 128 KiB and every four lines form a 0.5 MiB address-space block. 34
6.5 Physical layout of `lineitem.parquet` for the byte range from 0 B to 1.5 MiB, showing row group 0. Column chunks accessed by TPC-H Q21 (l_{suppkey}, l_{orderkey}, $l_{\text{receiptdate}}$, and $l_{\text{commitdate}}$) are shown in red, while all other column chunks are shown in light blue. Each line represents 128 KiB and every four lines form a 0.5 MiB address-space block. 35
6.6 Physical layout of `lineitem.parquet` for the byte range from 1.5 MiB to 3.0 MiB, showing row group 0. Column chunks accessed by TPC-H Q21 (l_{suppkey}, l_{orderkey}, $l_{\text{receiptdate}}$, and $l_{\text{commitdate}}$) are shown in red, while all other column chunks are shown in light blue. Each line represents 128 KiB and every four lines form a 0.5 MiB address-space block. 36
6.7 Warm execution time for parquet column and fixed-size caching with 64 threads, a block size of 512 KiB for fixed blocks, LRU replacement, a cache size of 20% of the dataset, and the TPC-H SF100 benchmark. Simulated storage has a bandwidth of 0.5 Gbit/s and a latency of 20 ms. 38
6.8 Warm execution time for fixed-size and column-chunk-based caching for sequences of three TPC-H queries, using 64 threads, a fixed block size of 512 KiB, LRU replacement, a cache size of 20% of the dataset, and the TPC-H SF100 benchmark. Simulated storage has a bandwidth of 0.5 Gbit/s and a latency of 20 ms. 41
6.9 Warm execution time for fixed-size and column-chunk-based caching for sequences of five TPC-H queries, using 64 threads, a fixed block size of 512 KiB, LRU replacement, a cache size of 20% of the dataset, and the TPC-H SF100 benchmark. Simulated storage has a bandwidth of 0.5 Gbit/s and a latency of 20 ms. 42
6.10 Warm execution time for LRU and Size-Aware LRU replacement policies with 64 threads, parquet column caching, a cache size of 20% of the dataset, and the TPC-H SF100 benchmark. Simulated storage has a bandwidth of 0.5 Gbit/s and a latency of 20 ms. 44
6.11 Cache hits for LRU and Size-Aware LRU replacement policies with 64 threads, parquet column caching, a cache size of 20% of the dataset, and the TPC-H SF100 benchmark. Simulated storage has a bandwidth of 0.5 Gbit/s and a latency of 20 ms. 45

6.12 Warm execution time for LRU and S3-FIFO replacement policies with 64 threads, parquet column caching, a cache size of 20% of the dataset, and the TPC-H SF100 benchmark. Simulated storage has a bandwidth of 0.5 Gbit/s and a latency of 20 ms. 47
6.13 Warm execution time for S3-FIFO promotion, S3-FIFO rounds, and S3-FIFO both with 64 threads, parquet column caching, a cache size of 20% of the dataset, and the TPC-H SF100 benchmark. Simulated storage has a bandwidth of 0.5 Gbit/s and a latency of 20 ms. 48
6.14 Warm execution time for S3-FIFO and S3-FIFO Rounds with 64 threads, parquet column caching, a cache size of 20% of the dataset, and the TPC-H SF100 benchmark. Simulated storage has a bandwidth of 0.5 Gbit/s and a latency of 20 ms. 49
6.15 Warm execution time for fixed-size lru and parquet column s3fifo_rounds caching with 64 threads, a block size of 512 KiB for fixed blocks, a cache size of 20% of the dataset, and the TPC-H SF100 benchmark. Simulated storage has a bandwidth of 0.5 Gbit/s and a latency of 20 ms. 52

List of Tables

6.1 Evaluation environment 28
6.2 Overall evaluation results 50

Bibliography

[Ama26] Amazon Web Services. *Amazon Simple Storage Service (Amazon S3)*. https://aws.amazon.com/s3/. Accessed September 21, 2026. 2026.

[Apa26] Apache Software Foundation. *Apache Parquet*. https://parquet.apache.org/. Accessed September 21, 2026. 2026.

[CI97] P. Cao and S. Irani. "Cost-Aware WWW Proxy Caching Algorithms." In: *Proceedings of the USENIX Symposium on Internet Technologies and Systems*. Monterey, CA: USENIX Association, Dec. 1997, pp. 193–206.

[den25] dentiny. *duck-read-cache-fs: A DuckDB Extension for Remote Filesystem Access Cache*. https://github.com/dentiny/duck-read-cache-fs. GitHub repository, accessed September 14, 2026. 2025.

[JZ02] S. Jiang and X. Zhang. "LIRS: An Efficient Low Inter-reference Recency Set Replacement to Improve Buffer Cache Performance." In: *Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*. 2002, pp. 31–42. DOI: 10.1145/511334.511340.

[Liu+25] C. Liu, A. Pavlenko, M. Interlandi, and B. Haynes. "Data formats in analytical DBMSs: performance trade-offs and future directions." In: *The VLDB Journal* 34.30 (2025). DOI: 10.1007/s00778-025-00911-1.

[MM03] N. Megiddo and D. S. Modha. "ARC: A Self-Tuning, Low Overhead Replacement Cache." In: *Proceedings of the 2nd USENIX Conference on File and Storage Technologies (FAST '03)*. San Francisco, CA: USENIX Association, Mar. 2003.

[Ope] OpenZFS. *OpenZFS: Adaptive Replacement Cache Implementation*. https://github.com/openzfs/zfs/blob/master/module/zfs/arc.c. GitHub repository, accessed September 16, 2026.

[RM19] M. Raasveldt and H. Mühleisen. "DuckDB: An Embeddable Analytical Database." In: *Proceedings of the 2019 International Conference on Management of Data*. SIGMOD '19. Association for Computing Machinery, 2019, pp. 1981–1984. DOI: 10.1145/3299869.3320212.

[Sta26] StarRocks. *Data Cache*. 2026. URL: https://docs.starrocks.io/docs/data_source/data_cache/ (visited on 08/21/2026).

[Tra26] Transaction Processing Performance Council. *TPC Benchmark H (TPC-H) Standard Specification*. https://www.tpc.org/tpc_documents_current_versions/pdf/TPC-H_v3.0.1.pdf. Version 3.0.1, accessed September 21, 2026. 2026.

[Yan+23] J. Yang, Y. Zhang, Z. Qiu, Y. Yue, and K. V. Rashmi. "FIFO Queues Are All You Need for Cache Eviction." In: *Proceedings of the 29th Symposium on Operating Systems Principles*. 2023. DOI: 10.1145/3600006.3613147.

[Yu25] X. Yu. "Disaggregation: A New Architecture for Cloud Databases." In: *Proc. VLDB Endow.* 18.12 (Aug. 2025), pp. 5527–5530. ISSN: 2150-8097. DOI: 10.14778/3750601.3760520.